

Chapter 4: Worked Answers

The Enhanced Entity-Relationship (EER) Model

Elmasri and Navathe, Fundamentals of Database Systems, seventh edition. Prepared for Database Management, WenYi Lee.

These are original worked answers, not an official publisher solution manual. Questions 4.1-4.33 follow the instructor's photographed copy, printed pp.165-176. The question statements and textbook figures are not reproduced. All example records are invented. A design answer is one defensible solution under its stated assumptions, not the only possible schema.

Reading This Answer Book

Page references without a qualifier refer to the photographed edition. Its pp.138-139 are missing from the scan; explanations of 4.1 use the complete same-edition textbook, printed pp.108-109, as a supplementary source. The other copy has different questions at 4.16, 4.18 and 4.25, and a different diagram at 4.26(a). Those versions do not determine the answers here.

ER/EER drawings use rectangles for entity types and diamonds for relationships. A double rectangle indicates a weak type; a double diamond identifies it through its owner. Keys and all attributes are supplied in the accompanying tables. Each ER (min,max) beside an entity counts relationships for one entity of that type. Repeated names in separate diagram panels denote the same type. For readability, some EER figures list properties inside the rectangles rather than drawing separate attribute ovals. A name marked `link:` denotes a relationship, not an attribute. Relationship panels show selected connections; the accompanying inventories supply the remaining attributes and constraints.

For specialization, `d` means disjoint and `o` means overlapping. A double line from the superclass to that circle means total; a single line means partial. The curved subset mark opens toward the superclass. A circle marked with a union symbol denotes a category, not a shared subclass. Figure captions state the constraints in words as well as symbols.

UML drawings are explicitly labeled. A multiplicity at the far end of a UML association counts the objects reachable at that end from one object at the other end. A hollow triangle points toward the superclass. Explicit `{complete, disjoint}` or `{incomplete, overlapping}` labels specify the generalization set. This uses OMG UML 2.5.1 rather than the textbook's filled-triangle convention; Answer 4.11 explains the difference.

Answers 4.28-4.33 include conceptual designs, checks and model-entry guidance. The required modeling-application work is **not completed**: no ERwin or Rational Rose session or native project file is supplied. The drawings are original illustrations, not screenshots from those applications. These answers do not change the course's assignments, scope or assessment policy.

4.1. Subclasses and Their Purpose

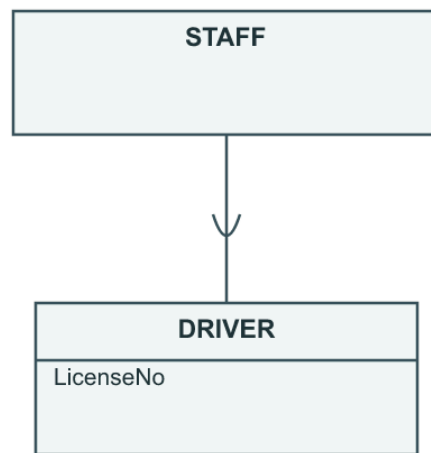
Source: 4.1-4.2.1, pp.140-142; supplementary textbook pp.108-109.

A subclass is a collection of entities that must also belong to a superclass. The subclass entity and superclass entity are the **same real-world object**. If S is a subclass of C, every member of S must be a member of C.

Use a subclass when some entities need attributes or relationships that do not apply to all members of the superclass. In this example all staff have an ID and name, but only drivers have a driving-license attribute and drive a vehicle.

A subclass preserves the entity's identity

EER



Every DRIVER is the same STAFF entity; not every STAFF member has to be a DRIVER.

Object	Superclass facts	Subclass facts
Staff S1, Alex	StaffId=S1; Name=Alex	DRIVER; LicenseNo=L1
Staff S2, Blair	StaffId=S2; Name=Blair	Not a driver in this example

Creating a separate DRIVER identity D1 and relating it to S1 would describe two objects unless an identity constraint is also supplied. EER inheritance already expresses that S1 in DRIVER is S1 in STAFF. Do not create a subclass just because a group has a convenient label; a simple attribute may be enough when there are no important local attributes or relationships (4.5.2).

4.2. EER Terms

Source: 4.1-4.4 and 4.5.3, pp.140-157; supplementary textbook pp.108-109.

Term	Meaning	Example
Superclass of a subclass	Class containing every entity of that subclass	STAFF contains every DRIVER.
Superclass/subclass relationship	Required inclusion with shared identity and inherited type	STAFF/DRIVER
IS-A relationship	Another name for a superclass/subclass relationship here	A DRIVER is a STAFF member.
Specialization	Define more specific subclasses from a superclass	Divide staff roles into DRIVER and CLERK.
Generalization	Identify common features and create a superclass for existing types	Generalize CAR and TRUCK into VEHICLE.
Category / union type	Class contained in the union of two or more defining classes	OWNER selected from PERSON or CORPORATION
Specific / local attribute	Attribute introduced at a subclass, not inherited from its superclass	DRIVER.LicenseNo
Specific relationship	Relationship restricted to a subclass as a participant	DRIVER drives VEHICLE.

An IS-A relationship is not an ordinary association such as STAFF works for DEPARTMENT: a staff member is not the same object as the department. A category does not require membership in all its defining classes.

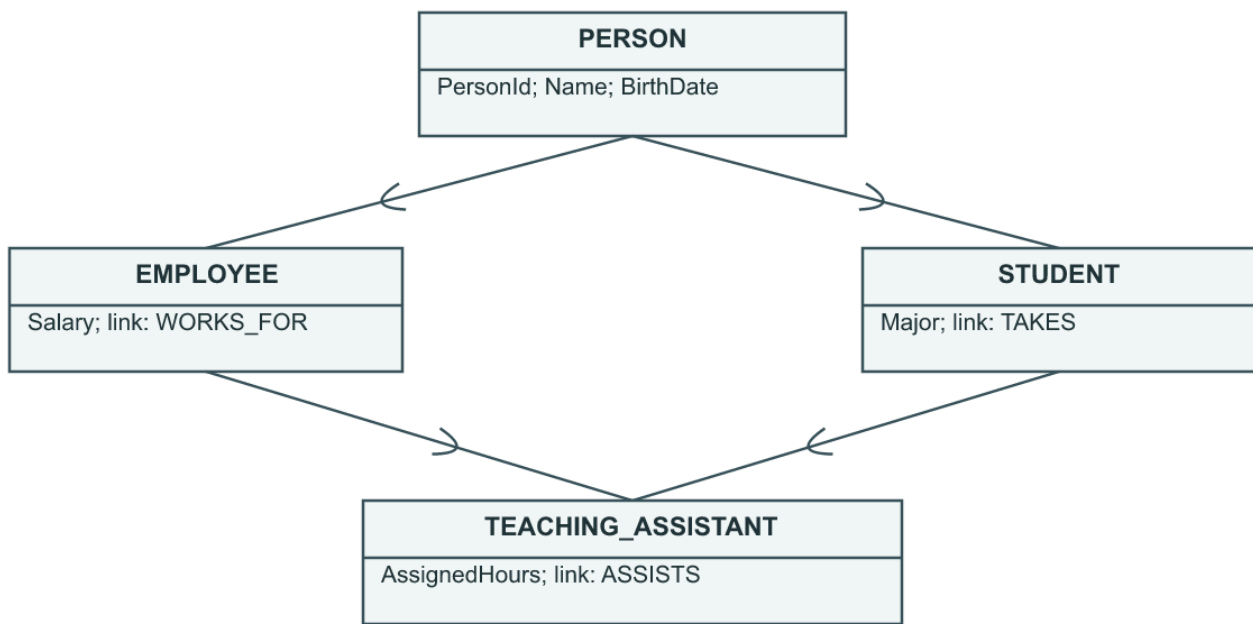
4.3. Attribute and Relationship Inheritance

Source: 4.1, p.140; 4.3.2, pp.146-149.

A subclass inherits its superclass's attributes and relationship participation. Inheritance continues through all ancestors. A teaching assistant who is both an employee and a student inherits from both, while PERSON attributes reached by two paths remain the same attributes, not two independent copies.

A shared subclass must belong to both parents

EER



PERSON's attributes are inherited once, although two inheritance paths reach them.

Source class	Inherited by TEACHING_ASSISTANT
PERSON	PersonId, Name, BirthDate
EMPLOYEE	Salary; WORKS_FOR department
STUDENT	Major; TAKES section
TEACHING_ASSISTANT itself	AssignedHours; ASSISTS section

If PersonId=P1 has Name=Alex through EMPLOYEE, it cannot simultaneously have Name=Blair through STUDENT in this model. Both paths describe P1. Inheritance avoids restating common definitions and permits constraints on all persons to apply to assistants. It does not say every employee-student must be an assistant: a shared subclass can be a proper subset of the intersection.

4.4. User-Defined and Predicate-Defined Subclasses

Source: 4.3.1, pp.143-144; formal definition 4.5.3, p.156.

Predicate-defined membership is determined exactly by a condition on superclass attributes. User-defined membership is assigned individually by an authorized user when no defining predicate has been specified.

Person	Age	Selected as mentor?	ADULT if Age $\geq$ 18	MENTOR, user-defined
P1	17	No	No	No
P2	24	No	Yes	No
P3	30	Yes	Yes	Yes

Membership comes from a rule or an individual decision

ADULT: predicate-defined	MENTOR: user-defined
Age $\geq$ 18, at a reference date P2 age 24: included P3 age 30: included	Recorded selection P2 not selected: excluded P3 selected: included

All three records are illustrative. The ADULT example assumes Age is known and evaluated at a declared reference date. P2 cannot be excluded from ADULT while Age=24 satisfies its definition. P2 may be excluded from MENTOR even though P2 is older than 18, because age is not the mentor-selection rule. User-defined does not mean unconstrained: MENTOR must still be a subset of PERSON.

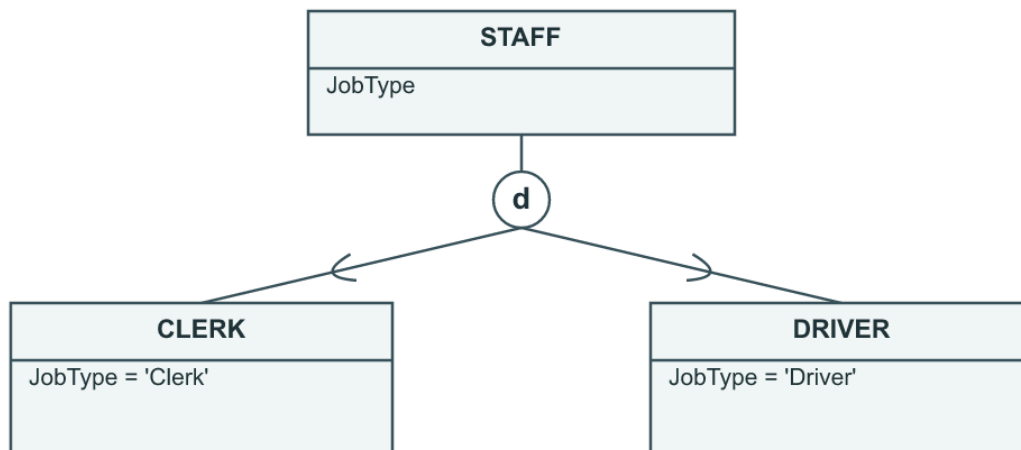
4.5. User-Defined and Attribute-Defined Specializations

Source: 4.3.1, pp.143-145; 4.5.3, p.156.

An attribute-defined specialization uses one defining superclass attribute for all its subclass membership tests. With a single-valued JobType and distinct values, JobType='Clerk' and JobType='Driver' cannot both hold for one person. The specialization is therefore disjoint. If JobType can also be 'Other', the two displayed subclasses are partial.

JobType defines two partial job subclasses

EER



JobType is single-valued; 'Other' is allowed outside the displayed subclasses.

Property	Attribute-defined	User-defined
How membership is determined	Evaluate the defining attribute	Record a user's classification
Example	JobType='Driver' defines DRIVER	Appoint staff to MENTOR and REVIEWER
Can it overlap?	Not for distinct equality tests on one single-valued attribute	Yes, unless a disjointness rule forbids it
Does it have to be total?	No; depends on the allowed domain and coverage	No; determined independently

A predicate-defined subclass may use a more general condition, such as **Age >= 18 AND TrainingComplete**. That is not automatically an attribute-defined specialization of the form used in this chapter.

4.6. The Two Independent Constraints

Source: 4.3.1, pp.144-146; 4.5.3, p.156.

Disjointness limits how many subclasses one entity may join. Completeness specifies whether every superclass entity must join at least one displayed subclass. Let **C** be a superclass and **A** and **B** its displayed subclasses.

Constraint combination	Permitted memberships per entity of C	Required set condition
Disjoint, total	Exactly one of A, B	No shared members; A union B equals C
Disjoint, partial	Zero or one	No shared members; union may omit members of C
Overlapping, total	One or both	A union B equals C
Overlapping, partial	Zero, one or both	Only A and B being subsets of C is required

Four combinations, with two displayed subclasses

Disjoint + total Exactly one membership Allowed: A only; B only Reject: neither; both	Disjoint + partial Zero or one membership Allowed: neither; A; B Reject: both
Overlap + total One or two memberships Allowed: A; B; both Reject: neither	Overlap + partial Zero, one or two memberships All four choices allowed All must still belong to C

Overlapping **permits** a shared member; it does not require one in every database state. Partial permits an uncovered member; it does not require one. A current example with no overlap does not prove a disjoint schema constraint.

Deleting an entity from C removes its membership in every subclass. Inserting into a total specialization requires at least one subclass membership in the resulting valid state. A predicate-defined subclass must contain exactly the entities satisfying its predicate, including after attribute updates.

4.7. Hierarchy Versus Lattice

Source: 4.3.2, pp.146-149, Figures 4.6-4.7.

A specialization hierarchy gives each subclass one immediate superclass. A specialization lattice permits a shared subclass with multiple immediate superclasses. This is the textbook's modeling use of the word lattice; it does not assert every mathematical lattice property for an arbitrary diagram.

Count immediate superclass links

Hierarchy	Lattice
PERSON > STUDENT STUDENT > GRAD_STUDENT One immediate parent per subclass	EMPLOYEE > STUDENT_ASSISTANT STUDENT > STUDENT_ASSISTANT Two immediate parents for one class

Here '>' means superclass of. A shared subclass inherits from every parent.

Structure	Example	Consequence
Hierarchy	PERSON > STUDENT > GRAD_STUDENT	One path of immediate parents
Lattice	STUDENT_ASSISTANT is both EMPLOYEE and STUDENT	Must meet both parent types and inherit both

Having two independent specializations of PERSON does not alone create a shared subclass. The connecting subclass must actually inherit from both parents. Conversely, merely belonging to two overlapping classes does not force the designer to create a separately named shared subclass.

4.8. Specialization Versus Generalization

Source: 4.2.2, pp.142-143; 4.3.3, pp.149-150.

Specialization starts with a broad type and defines narrower types: top-down conceptual refinement.

Generalization starts with types that have common properties and defines a common superclass: bottom-up conceptual synthesis.

The same final schema can be reached in two directions

Specialization: start broad	Generalization: find common facts
Start with VEHICLE Define CAR and TRUCK Keep common attributes in VEHICLE	Start with CAR and TRUCK Create common VEHICLE Move shared identifier and price up

For example, one designer starts with VEHICLE and discovers CAR and TRUCK; another starts with CAR and TRUCK and moves their shared identifier and price to VEHICLE. Both can reach the same final schema and constraints. This book does not draw different arrows for the two design processes because a finished schema describes the resulting structure, not the order of design decisions. Generalization often produces total coverage, but coverage must still be stated.

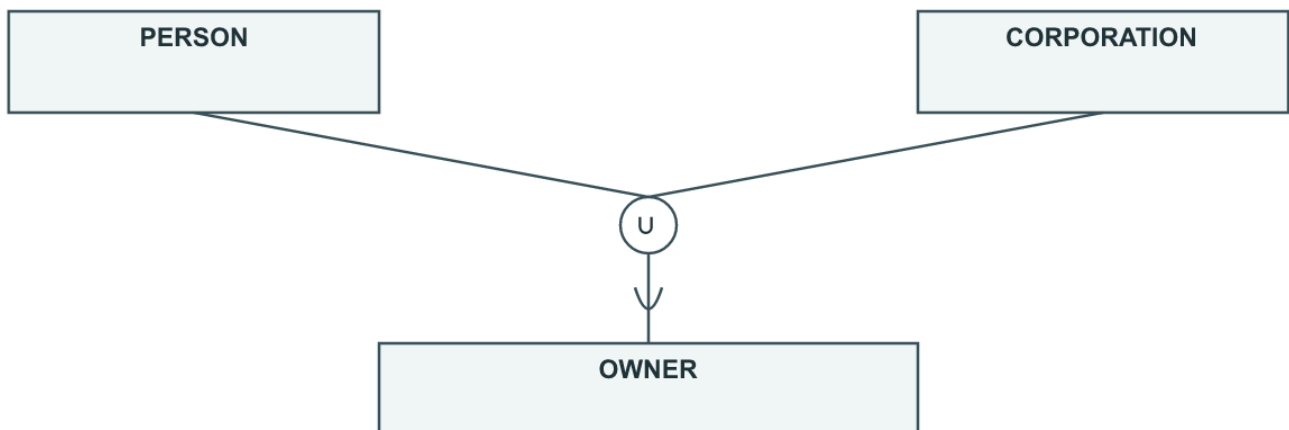
4.9. Category Versus Shared Subclass

Source: 4.4, pp.150-152, Figures 4.6 and 4.8; 4.5.3, pp.156-157.

An OWNER category can collect persons and corporations into one participant type for OWNS. A STUDENT_ASSISTANT shared subclass contains only entities that are both students and employees.

An owner can come from either defining class

EER



Question	OWNER category	STUDENT_ASSISTANT shared subclass
Required source membership	In PERSON or CORPORATION	In STUDENT and EMPLOYEE
Set rule	OWNER is a subset of the union	ASSISTANT is a subset of the intersection
Inheritance	From the defining class to which this owner belongs	From both parents
Example	Corporation C1 owns a plane	Person P2 is employed and enrolled

Here persons and corporations are assumed distinct kinds of objects. In the general set definition, union means membership in **at least one** defining class; exclusive membership needs disjoint source classes or an explicit rule. P1 who is only a student cannot be an assistant. Corporation C1 does not have to become a person to be an owner. A partial OWNER category also allows persons or corporations that own no represented plane to remain outside OWNER.

4.10. UML and EER Vocabulary

Source: 3.8, pp.115-118; 4.6-4.7, pp.157-163.

UML term in the question	Closest EER concept	Important qualification
Object	Entity	One instance, not a whole entity type
Class	Entity type / EER class	UML can also specify operations.
Association	Relationship type	Links instantiate it.
Aggregation	Whole-part relationship or composite structure	EER has no separate built-in aggregate-of-a-relationship construct.
Generalization	Superclass/subclass relationship	Subclass has the same entity identity.
Multiplicity	Cardinality and participation constraint	UML end placement differs from ER min-max placement.
Attributes	Attributes	A composite attribute may use a structured data type.
Discriminator	Defining attribute of specialization	In the book's qualified-association discussion, the word also refers to a partial-key qualifier. These contexts differ.
Link	Relationship instance	E.g., one particular employee-department association
Link attribute	Relationship attribute	UML commonly places it in an association class.
Reflexive association	Recursive relationship type	Role names distinguish the two ends.
Qualified association	Can represent an identifying relationship plus partial key	Qualification is more general than weak-entity identification.

For example, an EMPLOYEE's DependentName qualifier distinguishes dependents of that employee. The same name can appear under a different employee. A qualifier is not automatically a globally unique identifier.

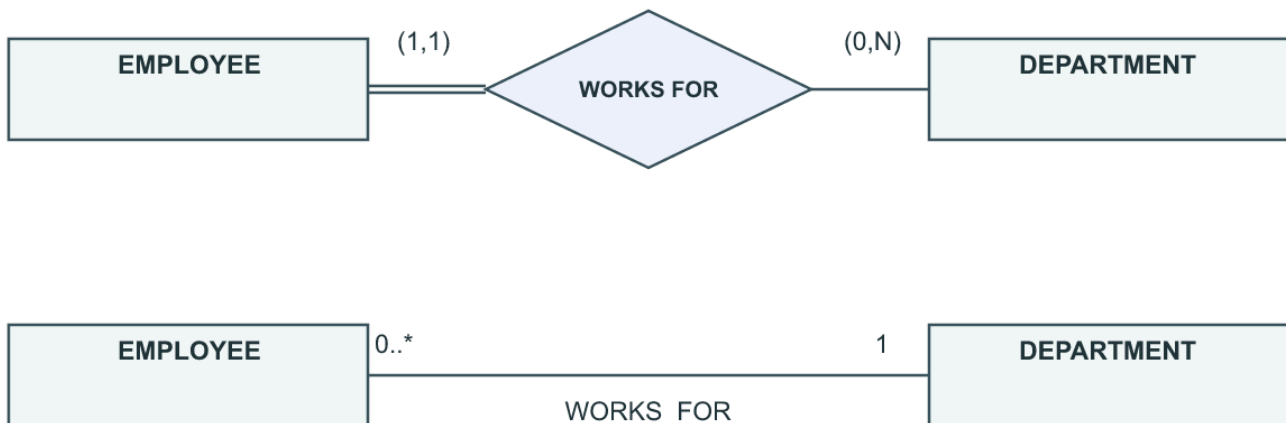
4.11. Comparing Diagram Notation

Source: 3.8, pp.115-118; 4.6, pp.157-158. UML clarification: [OMG UML 2.5.1](#), 9.2.4.2, 9.5.3 and 9.7.4, printed pp.102, 112 and 120-121.

Concept	EER in the textbook	UML used in these answers
Entity/class	Rectangle with external attribute ovals	Named rectangle with attribute and optional operation compartments
Relationship	Diamond and connecting edges	Association line; association class for its attributes
Counts	ER (min,max) beside the participating entity	min..max at the reached class end; * means unbounded
Key/weak entity	Underline; partial-key underline; double rectangle/diamond	Explicit uniqueness constraints; owner association and qualifier where useful
Inheritance	Subset edges and d/o circle	Hollow triangle pointing to superclass plus explicit constraint label
Total/partial	Double/single superclass-to-circle line	complete / incomplete generalization-set constraint
Whole-part	Usually an ordinary relationship plus rules	Shared aggregation or composition, with different semantics

One rule, two conventions for count placement

ER above; UML below



Both mean: each employee works for one department; a department can have zero or many employees.

The textbook's 4.6 uses triangle fill to distinguish overlapping and disjoint subclasses. OMG UML 2.5.1 uses a hollow generalization triangle and explicit generalization-set constraints. Do not infer disjointness from an unlabelled hollow triangle in a UML tool. The labeled notation in this answer book avoids that mismatch. Ordinary association also does not imply deletion of the related objects; Answer 4.25 separates association from composition.

4.12. Data Abstraction Concepts

Source: 4.7.1-4.7.4, pp.160-163.

Abstraction	What it does	EER representation / limitation
Classification	Groups similar instances by type	Entity types, relationship types, subclasses and categories
Instantiation	Creates or examines a particular instance	Entity and relationship instances; ordinary schema diagrams do not list them
Identification	Distinguishes constructs and real objects	Unique schema names; entity keys; owner plus partial key for weak types
Specialization	Defines a more specific type	Superclass/subclass relationship
Generalization	Defines a common broader type	Same relationship, reached from the other design direction
Aggregation	Forms a whole from components	Composite attributes or relationships; no direct relationship-as-object aggregation in this EER notation
Association	Relates independently identifiable objects	Relationship type

Schema inclusion differs from instance membership

Type relationship	Instance relationship
STUDENT is a subclass of PERSON Every student must be a person A schema constraint	P1 Alex is an instance of STUDENT A particular stored object Not a new subclass

STUDENT is a subclass of PERSON. Alex is an instance of STUDENT. Treating Alex as another subclass confuses the schema with its current data. A metaclass whose instances are themselves classes requires more than the book's basic two-level EER class/instance structure.

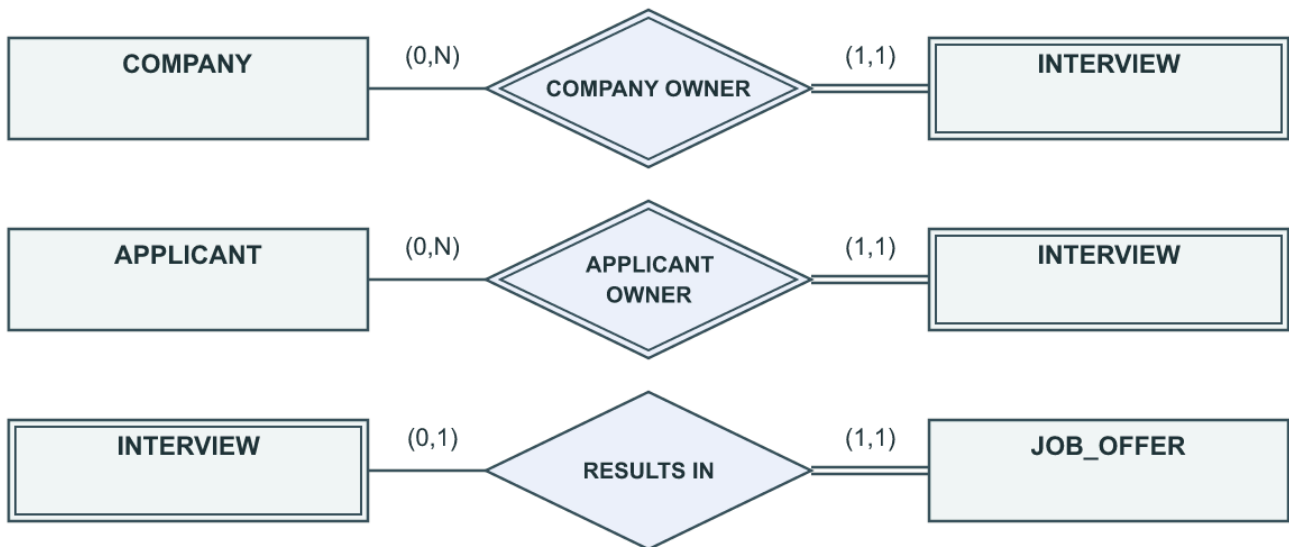
4.13. The Missing Aggregation Construct

Source: 4.7.4, pp.161-163, Figure 4.11.

The missing feature is treating one relationship instance together with its participants as a higher-level object that can participate in another relationship. An interview between an applicant and company may produce a job offer, but some interviews produce none.

An interview exists before any job offer

ER min-max participation



The two owners jointly identify INTERVIEW with its local InterviewNo.

Representation	Can retain an interview with no offer?	Reason
Only a ternary COMPANY-APPLICANT-OFFER relationship	No	A relationship tuple requires all three participants.
Connect one relationship diamond to another	Not valid in this EER notation	A relationship is not an entity participant.
Add INTERVIEW as an entity	Yes	INTERVIEW exists independently of RESULTS_IN.
Extend EER with an aggregate-object construct	Yes	The aggregate interview can be related to an offer.

For the concrete ER solution, use INTERVIEW identified by CompanyId, ApplicantId and InterviewNo; InterviewNo is local to that pair. Store Date, ContactName and ContactPhone on INTERVIEW. Assume one interview produces at most one offer, each offer comes from one interview, and OfferId identifies JOB_OFFER. The pair of owners and InterviewNo must all be present. This weak entity is a representational alternative; it does not add aggregation syntax to the original EER model.

4.14. Conceptual Modeling and Knowledge Representation

Source: 4.7, pp.158-160.

Both describe a domain using concepts, relationships and constraints, selecting relevant properties while suppressing irrelevant detail. Both can use classification, inheritance and explicit rules.

Comparison	Conceptual database modeling	Knowledge representation
Main emphasis in this chapter	Data to store and constraints on valid states	Knowledge to represent and reason with
Common result	Schema for a defined application	Domain model with facts and inference rules
Exceptions and defaults	Often need explicit schema constructs and constraints	May explicitly support defaults, exceptions and incomplete knowledge
Levels	EER primarily separates classes and instances	Some approaches also treat classes as instances of metaclasses
Example	Record P1 as a student in a department	Infer P1 is a person from student membership and an IS-A rule

This is a comparison of emphasis, not a claim that database systems cannot perform inference or that every KR implementation is more expressive or slower. The textbook itself discusses extensions of database technology. A missing fact is not automatically false in every representation; that depends on the chosen semantics, which must be stated before drawing conclusions.

4.15. Ontology and Database Schema

Source: 4.7.5, pp.163-164.

Both can describe entities or concepts, attributes, relationships and specializations. A database schema usually targets the data requirements of one application. An ontology typically aims at a shared account of a domain's concepts and relationships, possibly with axioms and inference.

Example question	Application schema	Domain ontology
What is a course?	COURSE(Courseld, Title, Credits) in this system	Shared meaning and relationships of the concept Course
What must be checked?	Every SECTION references an existing COURSE.	A taught course is an educational offering under stated definitions.
What is shared?	Structure and constraints for stored data	Vocabulary and meaning across participating systems

A schema may form part of an ontology, and an ontology may guide a schema. They are not mutually exclusive categories. Nor does an ontology have to describe all reality. Its domain, intended users and reasoning assumptions still need boundaries. A shared label alone does not ensure two systems mean the same thing by that label.

4.16. Constraints on a Specialization

Source: photographed exercise 4.16, p.166; 4.3.1 and 4.5.3. This is the photographed question about constraints, not the other copy's open-ended five-entity design exercise.

Every subclass must be a subset of its superclass. Then specify disjointness, completeness and any defining membership predicates. In this example STAFF is specialized into SALARIED and HOURLY, using a single-valued PayMethod whose allowed values are exactly 'Salaried' and 'Hourly'. The specialization is attribute-defined, disjoint and total.

Check each rule independently

Valid: S1 STAFF, PayMethod='Hourly' Only in HOURLY One matching subclass	Invalid: two subclasses HOURLY and SALARIED Disjointness fails
Invalid: no subclass STAFF, but neither pay subclass Totalness fails	Invalid: missing superclass HOURLY S9, no STAFF S9 Inclusion fails

Proposed state	Decision and reason
S1 is staff, PayMethod='Hourly', only in HOURLY	Valid
S1 is in both HOURLY and SALARIED	Invalid: violates disjointness
S1 is staff but in neither subclass	Invalid: violates totalness
S9 is HOURLY but not STAFF	Invalid: violates subclass inclusion
S1 has PayMethod='Hourly' but is only SALARIED	Invalid: violates the defining predicates

Changing PayMethod requires the corresponding membership change in the same valid resulting state. Removing the only subclass membership is not allowed while leaving the entity in a total superclass. For an overlapping partial example instead, let STAFF have MENTOR and REVIEWER roles: a staff member may hold neither, either or both. The four combinations in Answer 4.6 are independent of whether membership is assigned by users or by a predicate.

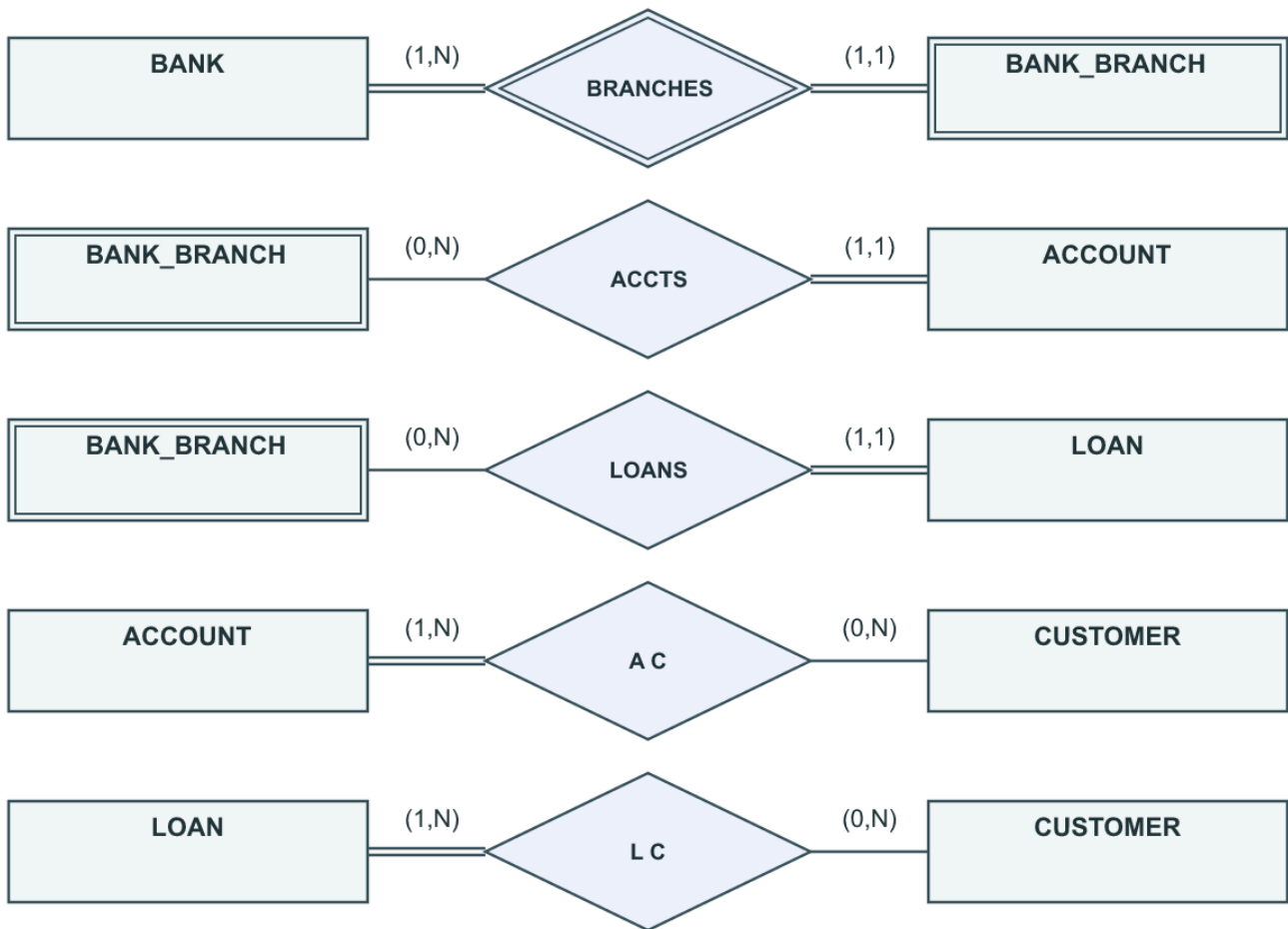
4.17. BANK with Account, Loan and Event Subclasses

Source: photographed exercise 4.17, p.166; BANK Figure 3.22, p.129. The question says Figure 3.21, but that figure is AIRLINE in both supplied copies. The BANK diagram is Figure 3.22; it is the base used here.

Keep the original bank, branch, customer, account and loan relationships. Add account and loan specializations and owner-dependent event records. Assume account and loan numbers are globally unique as in the base figure; event sequence numbers are only unique within their account or loan.

BANK relationships retained from Figure 3.22

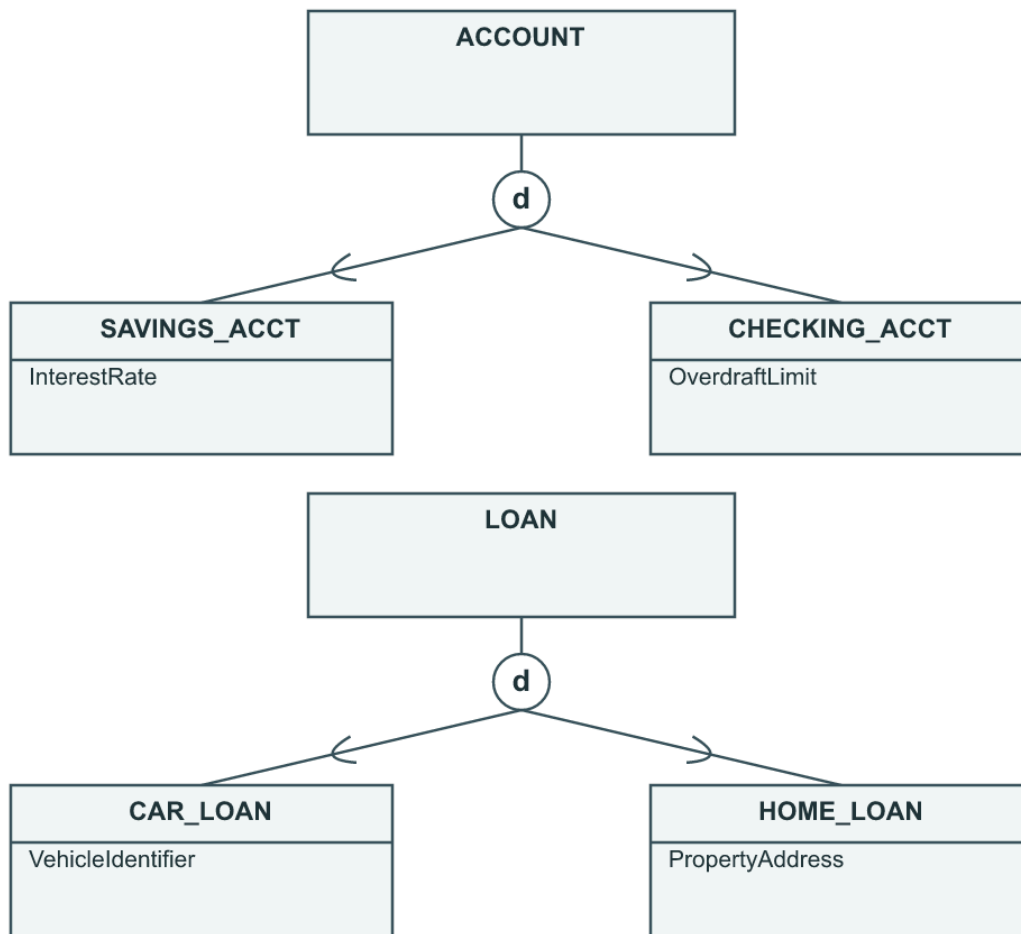
ER min-max participation



Type	Key / partial key	Attributes
BANK	Code	Name, Addr
BANK_BRANCH, weak	BANK.Code + Branch_no; Branch_no partial	Addr
CUSTOMER	Ssn	Name, Addr, Phone
ACCOUNT	Acct_no	Balance; AccountType
SAVINGS_ACCT	Inherits Acct_no	InterestRate
CHECKING_ACCT	Inherits Acct_no	OverdraftLimit
LOAN	Loan_no	Amount; LoanType
CAR_LOAN	Inherits Loan_no	VehicleIdentifier
HOME_LOAN	Inherits Loan_no	PropertyAddress
TRANSACTION, weak	Acct_no + TxnNo; TxnNo partial	Amount, Date, Time, Kind
PAYMENT, weak	Loan_no + PaymentNo; PaymentNo partial	Amount, Date, Time

Account and loan classifications are independent

EER

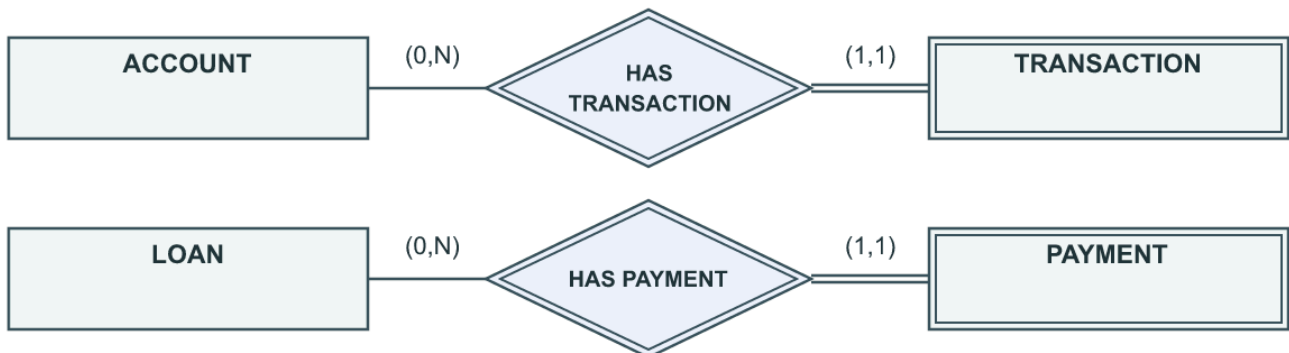


Both specializations are partial: other account and loan types may exist.

Account and loan specializations are disjoint but partial: the question's ellipsis permits other types. Define the displayed memberships by **AccountType** and **LoanType** values. Amount, Date and Time are event facts, not an account's or loan's single attributes. Timestamps are not assumed unique.

Each account or loan owns a sequence of events

ER min-max participation

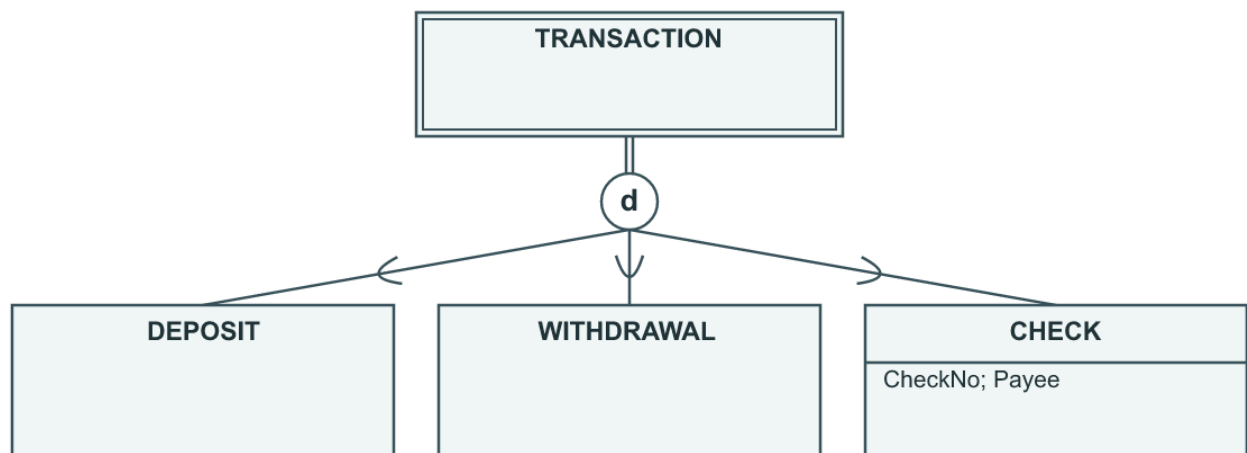


TRANSACTION kinds: DEPOSIT / WITHDRAWAL / CHECK, total and disjoint in this example.

Assume each recorded transaction is exactly one of DEPOSIT, WITHDRAWAL or CHECK; this event specialization is total and disjoint. CHECK is a separate kind here, not simultaneously a WITHDRAWAL subclass. It adds CheckNo and Payee; the other two need no extra attributes. If other transaction kinds are required, add them or make this event specialization partial. Each PAYMENT belongs to exactly one LOAN; each loan may have no payments yet. A transaction belongs to exactly one ACCOUNT; each account may have no transactions yet.

Transaction kinds inherit the account-local identifier

EER



Kinds form a total classification only under this answer's explicitly restricted event domain.

Account	TxnNo	Kind	Amount	Date	Time
A1	1	DEPOSIT	100	2026-01-02	10:00:00
A1	2	WITHDRAWAL	30	2026-01-02	10:00:00
A2	1	DEPOSIT	70	2026-01-02	10:00:00

All three identities are distinct. Under the illustrative signed-effect rule, the first two events change A1's balance by +70, before any fees or interest. They do not establish its absolute balance without an opening balance. Require positive event amounts and a valid owner. Do not infer transaction history or loan amortization solely from the current Balance or Amount attribute.

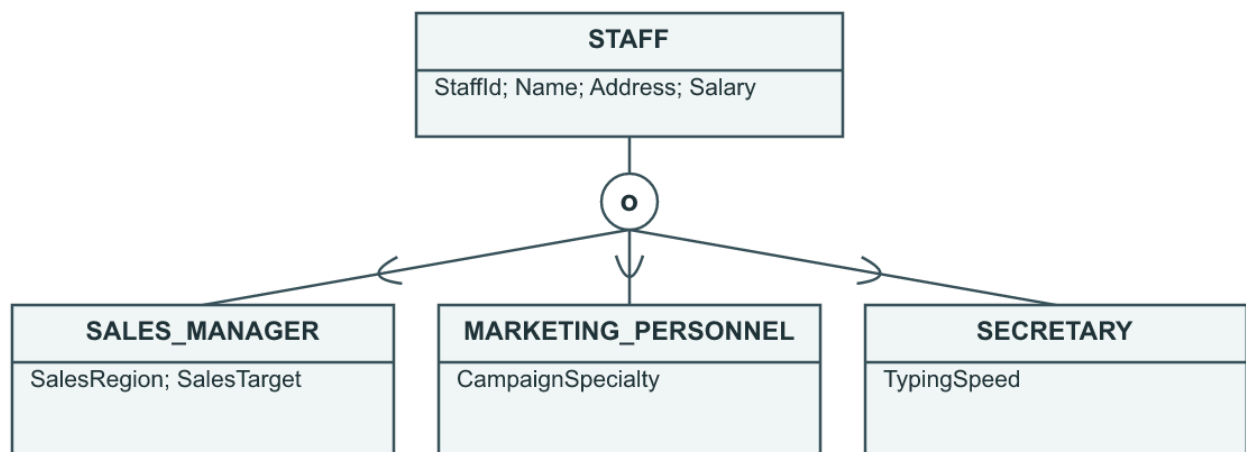
4.18. STAFF and Specialized Job Roles

Source: photographed exercise 4.18, p.166; 4.1-4.3. This is the staff-role question, not the other copy's Olympic-facilities problem.

STAFF is the superclass. SALES_MANAGER, MARKETING_PERSONNEL and SECRETARY are subclasses with shared staff identity. The question gives no exclusivity or coverage rule, so this solution explicitly permits overlapping, partial roles.

STAFF with three possible job roles

EER



Overlapping and partial: a staff member may hold neither, one or several of these roles.

Class	Identifier	Attributes introduced here
STAFF	StaffId	Name, Address, Salary
SALES_MANAGER	Inherits StaffId	SalesRegion, SalesTarget
MARKETING_PERSONNEL	Inherits StaffId	CampaignSpecialty
SECRETARY	Inherits StaffId	TypingSpeed

For example, S1 Alex is both SALES_MANAGER and MARKETING_PERSONNEL. S1 has one Name and Salary inherited from STAFF, plus SalesRegion='North', SalesTarget=500 and CampaignSpecialty='Email'. S2 Blair is SECRETARY and has TypingSpeed=70 words per minute. S3 Casey may be staff in none of these three roles. These attribute values and role rules are teaching assumptions, not measurements or universal job definitions.

Do not make SALES_MANAGER a subclass of MARKETING_PERSONNEL merely because the roles sound related. That extra inclusion would require every sales manager to be marketing personnel, which the question does not say. A STAFF subtype inherits shared attributes; it is not related to a separate STAFF person by an ordinary employment relationship.

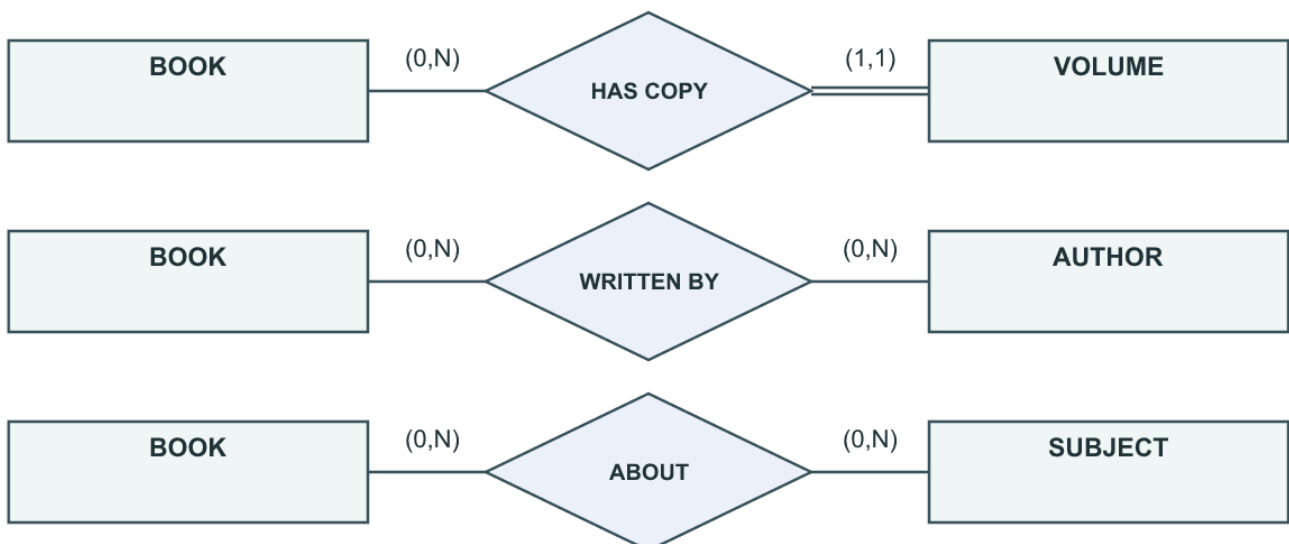
4.19. Library: Titles, Copies, Members and Loans

Source: photographed exercise 4.19, pp.167-168; abstractions in 4.7.

Separate a bibliographic book record from each physical copy and from each borrowing event. Under the question's ISBN assumption, ISBN identifies a particular edition/binding/language, not its title text. Two copies of that edition share ISBN but have different Volumelds. An acquisition wish can exist even when the library owns no copy.

Separate a bibliographic record from its physical copies

ER min-max participation



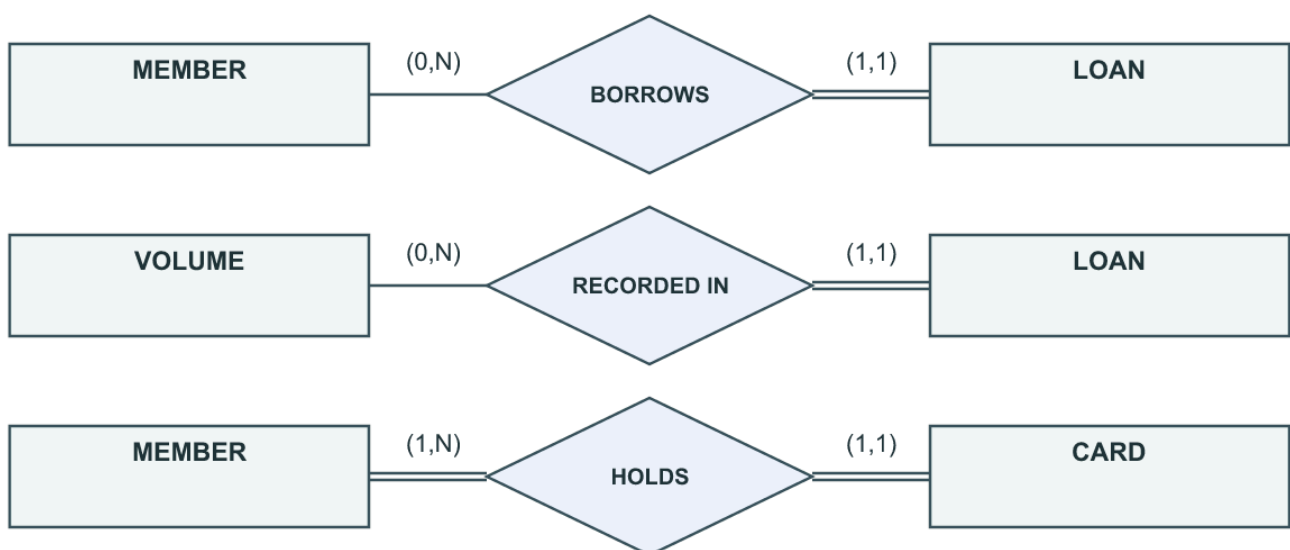
A desired ISBN may be represented even when the library owns no volume.

Type	Key	Attributes / facts
BOOK	ISBN, as assumed by the question	Title, Description, Edition, Language, Binding; LendingClass
VOLUME	Volumeld	ShelfLocation, CopyStatus: available/on-loan/lost/destroyed
AUTHOR	AuthorId, added identifier	Name
SUBJECT	SubjectId, added identifier	Label
ACQUISITION_WISH	ISBN, inherited from BOOK	Reason, DateListed; subset of BOOK
MEMBER	MemberId, added stable identifier; Ssn unique	Name, CampusAddress, HomeAddress, set of Phone values
PROFESSOR	Inherits MemberId	EmployeeId, unique for the source employee record
CARD	CardNo	Photo, IssuedDate, ExpiryDate
LOAN	LoanId, added identifier	CheckoutDate, DueDate, ReturnedDate optional
NOTICE	NoticeId	NoticeType, SentDate, Destination
LIBRARY_STAFF	StaffId	Name, set of Role values

Role values cover chief librarian, departmental associate librarian, reference librarian, check-out staff and library assistant. A multivalued role attribute is enough here because the case gives no role-specific stored attributes or relationships. AUTHOR and SUBJECT are separate types for catalog lookup.

A loan is one borrowing event

ER min-max participation



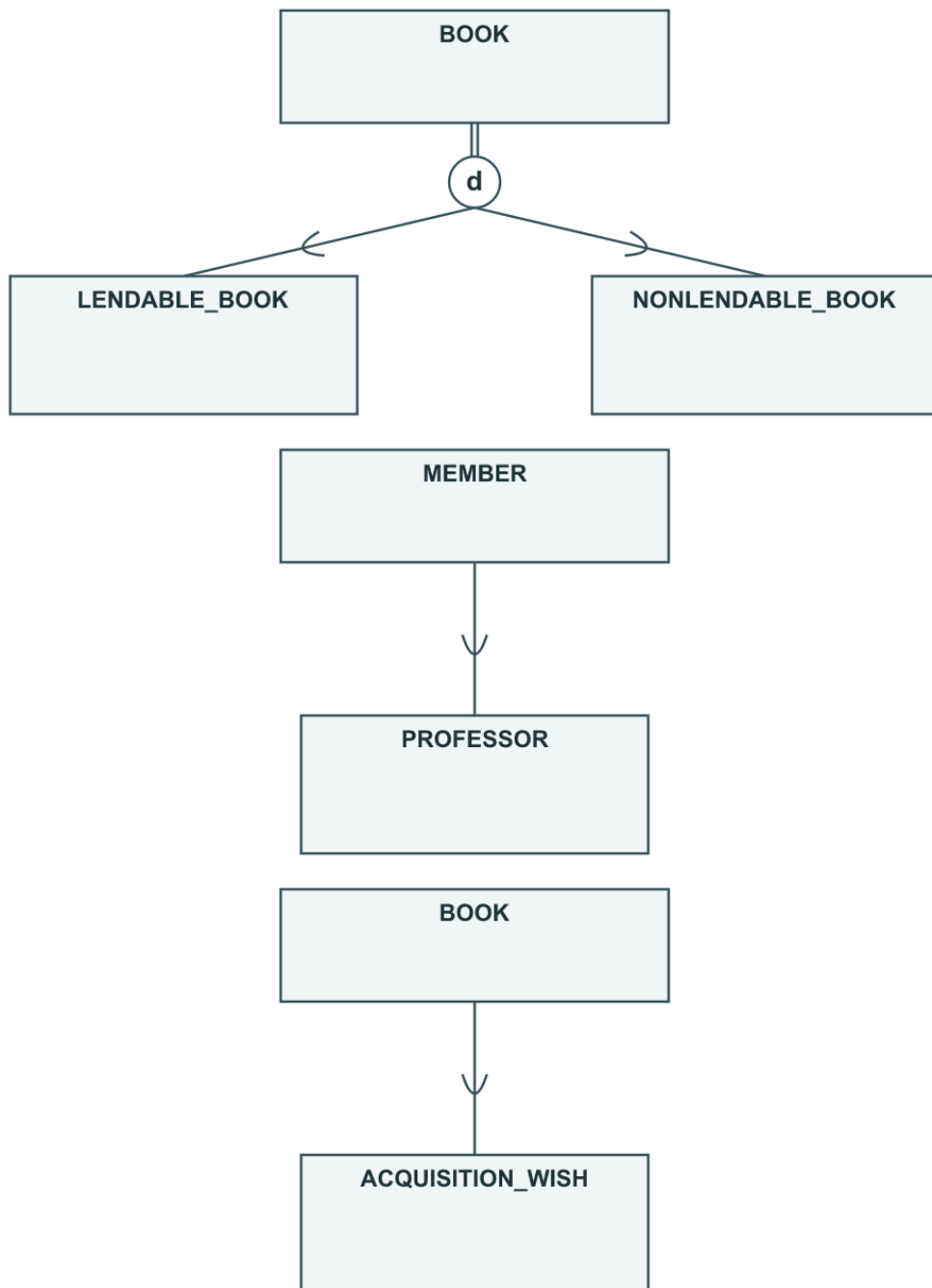
Five limits open loans, not lifetime LOAN records. Card history is retained.

Relationship	Participation of first type	Participation of second type
BOOK-HAS_COPY-VOLUME	BOOK (0,N)	VOLUME (1,1)
BOOK-WRITTEN_BY-AUTHOR	BOOK (0,N), allow unknown authors	AUTHOR (0,N)
BOOK-ABOUT-SUBJECT	BOOK (0,N)	SUBJECT (0,N)
MEMBER-HOLDS-CARD	MEMBER (1,N), card history retained	CARD (1,1)
MEMBER-BORROWS-LOAN	MEMBER (0,N), lifetime history	LOAN (1,1)
VOLUME-RECORDED_IN-LOAN	VOLUME (0,N)	LOAN (1,1)
MEMBER-RECEIVES-NOTICE	MEMBER (0,N)	NOTICE (1,1)
NOTICE-CONCERNS-LOAN	NOTICE (0,1)	LOAN (0,N)

Use PROFESSOR as a partial subclass of MEMBER. Split BOOK into LENDABLE_BOOK and NONLENDABLE_BOOK as a total, disjoint classification, with a nonlending reason such as reference, rare, or map. ACQUISITION_WISH is a separate partial subclass, not an alternative to those two: a desired rare book may be both a wish and nonlendable. Catalog inclusion does not imply physical availability.

Library classifications answer separate questions

EER



PROFESSOR and ACQUISITION_WISH are independent partial subclasses of different superclasses.

Semantic constraints, beyond local cardinalities:

1. An outstanding loan has no ReturnedDate. At most one outstanding loan may reference a volume. A member has at most five outstanding loans. This solution applies that limit to professors too because the text changes their loan duration, not their count limit.
2. Only an available volume of a LENDABLE_BOOK may be checked out. Lost and destroyed copies remain in history but are not available. Counts of owned, available and loaned copies are derived separately.

3. Ordinary loans are due 21 days after checkout. Professor loans are due three calendar months after checkout. The one-week/two-week grace periods affect reminder timing, not the due date. Use a declared month-end convention for dates such as November 30; do not silently replace three months with 90 days.
4. A card is valid for four years. Retain old cards but allow only one active card per member in this design. Renewal notices are sent one calendar month before expiry. Professor notices use CampusAddress; record the actual destination sent to rather than relying on an address that can later change.
5. Each professor in the institute's employee feed must have one matching membership; account creation and mailing the card are required operations, not something subclass notation executes automatically.
6. A return reminder concerns a loan of the same recipient. A renewal notice has no loan link and refers to the member's expiring card; store that CardNo as an additional optional NOTICE-CARD relationship. Require exactly the appropriate target for each notice type.
7. ACTIVE_MEMBER may be a predicate-defined subclass using a derived BorrowCountForYear ≥ 10 at a specified reference year. The count measures borrowing events, not how many books are currently out.

Example with symbolic ISBN values	What follows
BOOK B1; copies V1 and V2; only V1 is on loan	Two owned copies; one on loan; one available if V2 is lendable and not lost
BOOK B2; acquisition wish; no volume	A valid catalog/wish record, not a physical book that can be lent
MEMBER M1 has five open loans and twenty returned loans	May retain all twenty-five; a sixth open loan is rejected

Abstractions: BOOK, MEMBER and LOAN provide classification; BORROWS and HAS_COPY classify associations. ISBN, VolumeId and LoanId provide identification at different levels. PROFESSOR/MEMBER provides specialization. Name/address components are attribute aggregation; combining a member, volume and checkout facts into a LOAN entity makes the borrowing event identifiable. A volume is not a subclass of BOOK: it is a physical copy associated with a bibliographic record. A professor is the same person as their member record.

Implementation/workload information, not conceptual cardinality rules: the approximate member/title/volume totals, loan proportions, return behavior, borrowing concentration, description length and machine-readable card format affect capacity, indexing, interfaces and notices. They do not impose a fixed 2.5-copy cardinality, require 10% of copies always on loan, or make overdue items automatically lost. The quoted institution is the textbook's case study, not a verified description of its current policies or operations.

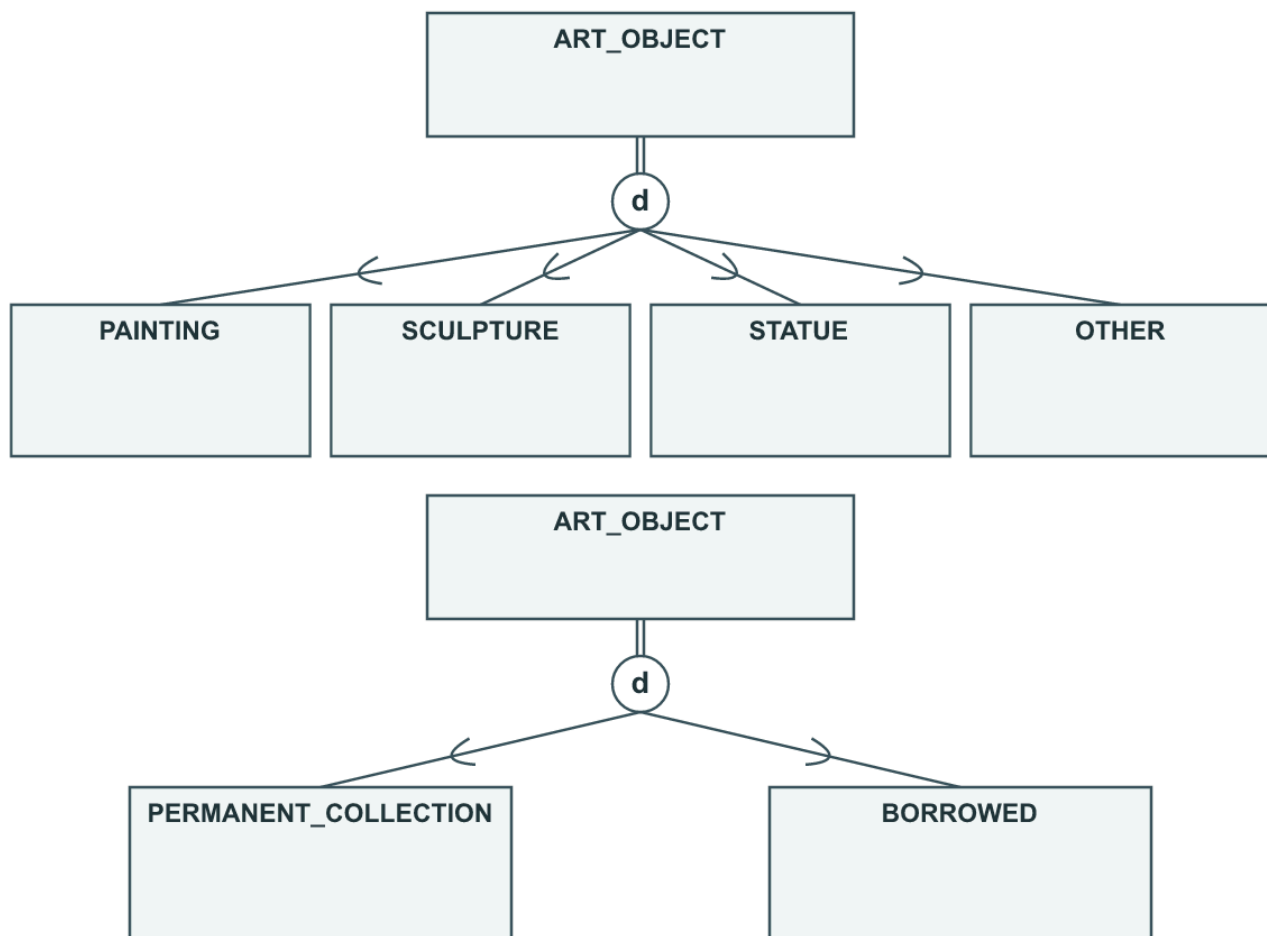
4.20. Art Museum

Source: photographed exercise 4.20, pp.168-169.

Use two independent total, disjoint specializations of ART_OBJECT: one for its cataloged art type and one for whether the museum owns or borrows it. All subclasses retain the same Id_no. An object must have one art type and one ownership classification; those classifications are not alternatives to each other.

Art type and ownership are separate total classifications

EER

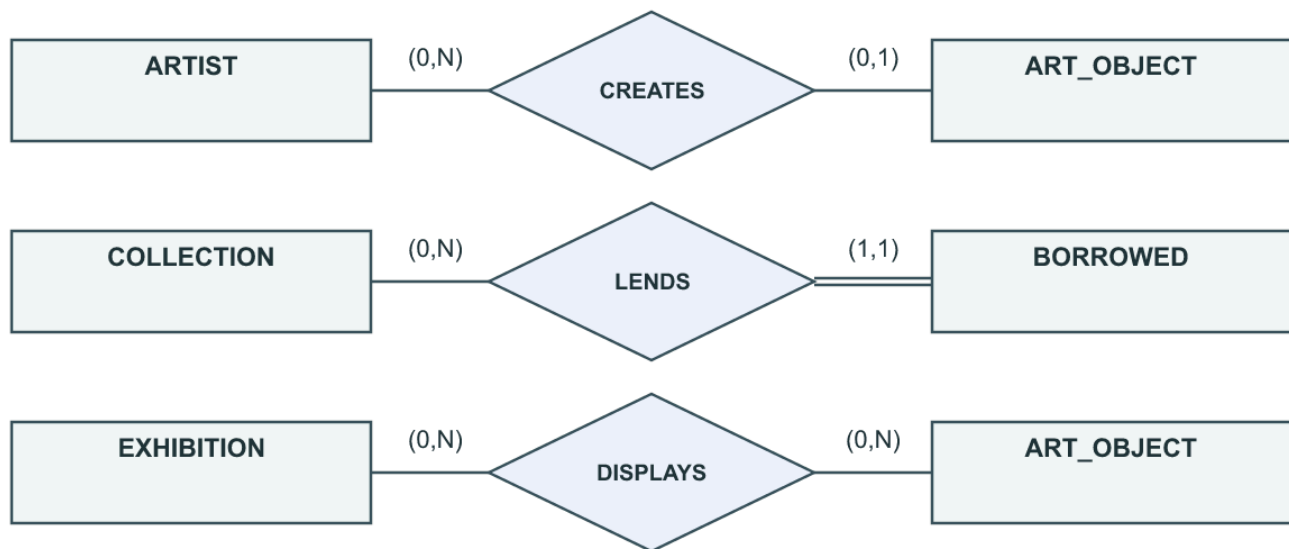


An object belongs to exactly one subclass in each group, under the stated cataloging rule.

Type	Key	Attributes introduced here
ART_OBJECT	Id_no	Year optional, Title, Description, Origin, Epoch
PAINTING	Inherits Id_no	Paint_type, Drawn_on, Style
SCULPTURE	Inherits Id_no	Material, Height, Weight, Style
STATUE	Inherits Id_no	Material, Height, Weight, Style
OTHER	Inherits Id_no	Type, Style
PERMANENT_COLLECTION	Inherits Id_no	Date_acquired, Status, Cost
BORROWED	Inherits Id_no	Date_borrowed, Date_returned optional
ARTIST	Name, unique by the question	DateBorn optional, Date_died optional, Country_of_origin, Epoch, Main_style, Description
EXHIBITION	ExhibitionId, added identifier	Name, Start_date, End_date
COLLECTION	Name, unique by the question	Type, Description, Address, Phone, Contact_person

Museum relationships retain unknown artists and repeat exhibitions

ER min-max participation



Relationship	First participant	Second participant
ARTIST-CREATES-ART_OBJECT	ARTIST (0,N)	ART_OBJECT (0,1)
COLLECTION-LENDs-BORROWED	COLLECTION (0,N)	BORROWED (1,1)
EXHIBITION-DISPLAYS-ART_OBJECT	EXHIBITION (0,N)	ART_OBJECT (0,N)

Assumptions: one cataloged artist per object; an unknown artist is absence of the association, not a fabricated ARTIST named 'Unknown'. Exhibitions can be planned before objects are assigned, and names need not be unique. The art-type classification is the museum's mutually exclusive cataloging rule for this exercise; it is not a general claim that statues cannot be sculptures. OTHER makes the classification exhaustive.

This is a snapshot of ownership and one recorded borrowing episode per object. An owned object currently on loan remains PERMANENT_COLLECTION with Status='on loan'; it does not become BORROWED. BORROWED means obtained from another collection, not necessarily still physically present. Preserve return dates. For multiple historical borrowing episodes, replace the two borrowing dates by a BORROWING record with its own identifier; that is an extension to this scope.

Example	Valid?
O1: PAINTING and PERMANENT_COLLECTION, Cost=200, Status='stored'	Yes
O2: STATUE and BORROWED from collection C1, return not yet recorded	Yes
O3: PAINTING and OTHER at the same time	No, violates the selected art-type rule
O2 borrowed without any lender	No, violates LENDS minimum one

Check nonnegative cost, positive measured height/weight when applicable, Start_date <= End_date, and Date_borrowed <= Date_returned when returned. These value checks supplement the diagram; they do not follow from an M:N line.

4.21. SMALL_AIRPORT in UML

Source: photographed exercise 4.21 and Figure 4.12, pp.169-170; 3.8 and 4.6.

Translate the entity types into classes, keep PERSON inheritance for PILOT and EMPLOYEE, and retain all seven associations. PILOT and EMPLOYEE are independent partial subclasses: a person can be neither, either or both. The question expressly exempts mapping the OWNER category to UML. OWNER is therefore shown as a class with its union membership stated in text, not falsely drawn as inheriting from both PERSON and CORPORATION.

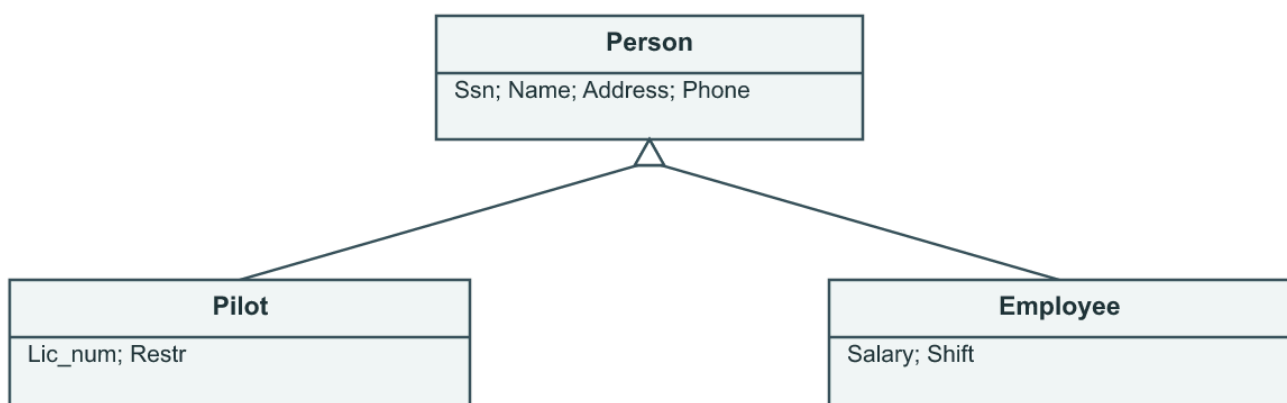
Airport UML class properties

UML property inventory

Person Ssn {unique} Name; Address; Phone	Pilot / Employee Both specialize Person Pilot: Lic_num; Restr Employee: Salary; Shift
Airplane / PlaneType Airplane: RegNo {unique} PlaneType: Model {unique} Capacity; Weight	Hangar Number {unique} Capacity; Location
Service Date; Workcode; Hours Unique per Airplane + Date + Workcode	Owner / Corporation Owner category mapping exempt Corporation: Name {unique} Address; Phone

Airport UML: two independent PERSON subclasses

UML

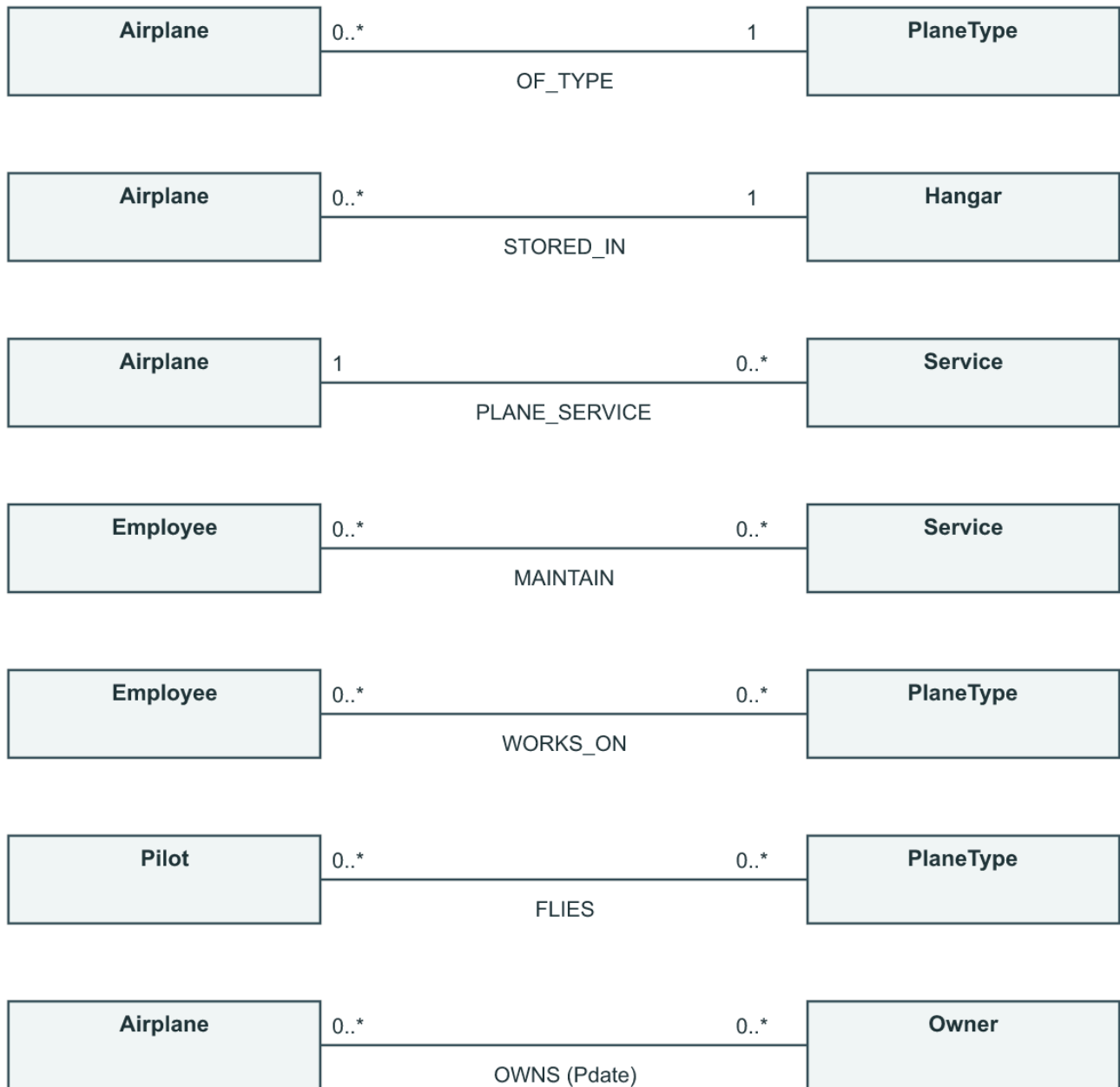


{incomplete, overlapping}: a person can be a pilot, an employee, both, or neither.

Class	Identifier	Properties
Person	Ssn	Name, Address, Phone
Pilot	Inherits Ssn	Lic_num, Restr
Employee	Inherits Ssn	Salary, Shift
Corporation	Name	Address, Phone
PlaneType	Model	Capacity, Weight
Airplane	RegNo, corresponding to Reg#	No additional properties required
Hangar	Number	Capacity, Location
Service	RegNo + Date + Workcode	Date, Workcode, Hours
Owner	Identity inherited selectively from Person or Corporation	Category mapping exempt here
Ownership, association class	Airplane-Owner pair	Pdate

Airport UML association ends

UML end multiplicities



Pdate is the Ownership association-class property. Service's composite key is in the table.

Association	Objects reached from one left object	Objects reached from one right object
Airplane-OF_TYPE-PlaneType	1 PlaneType	0..* Airplanes
Airplane-STORED_IN-Hangar	1 Hangar	0..* Airplanes
Airplane-PLANE_SERVICE-Service	0..* Services	1 Airplane
Employee-MAINTAIN-Service	0..* Services	0..* Employees
Employee-WORKS_ON-PlaneType	0..* PlaneTypes	0..* Employees
Pilot-FLIES-PlaneType	0..* PlaneTypes	0..* Pilots
Airplane-OWNS-Owner	0..* Owners	0..* Airplanes

OF_TYPE and STORED_IN use the narrative's exactly one. Other zero minima preserve the figure's single participation lines; the text's word 'many' does not specify a precise positive minimum. OWNS retains its M:N maximum, allowing co-ownership. Pdate belongs to Ownership, not to Owner alone.

For Service, model a normal owner association plus the explicit uniqueness constraint (Airplane.RegNo, Date, Workcode). Alternatively, qualify the Airplane end with (Date, Workcode), yielding 0..1 Service for an airplane and those qualifier values. Unqualified, one airplane can have many services. Two services on different airplanes may have the same date and work code. Two on the same airplane with the same pair would collide under the book's key.

No aviation safety or authorization procedure is implemented. A further business constraint could require an employee maintaining a plane to be qualified for its type; Figure 4.12's three separate relationships alone do not enforce that cross-relationship rule. Do not invent it as a drawn fact.

4.22. UNIVERSITY Figure 4.9 in UML

Source: photographed exercise 4.22, p.170; Figure 4.9, p.153 and 4.5.1, p.154. This is **not** the different UNIVERSITY schema in Chapter 3.

Map the PERSON specialization, student and section predicates, all ordinary entity attributes, and fourteen relationships. As allowed in 4.21, the category construction for INSTRUCTOR_RESEARCHER is exempt; keep that participant class for TEACH and SUPPORT, and state its intended membership separately.

UNIVERSITY: superclass and predicate-defined classes

UML

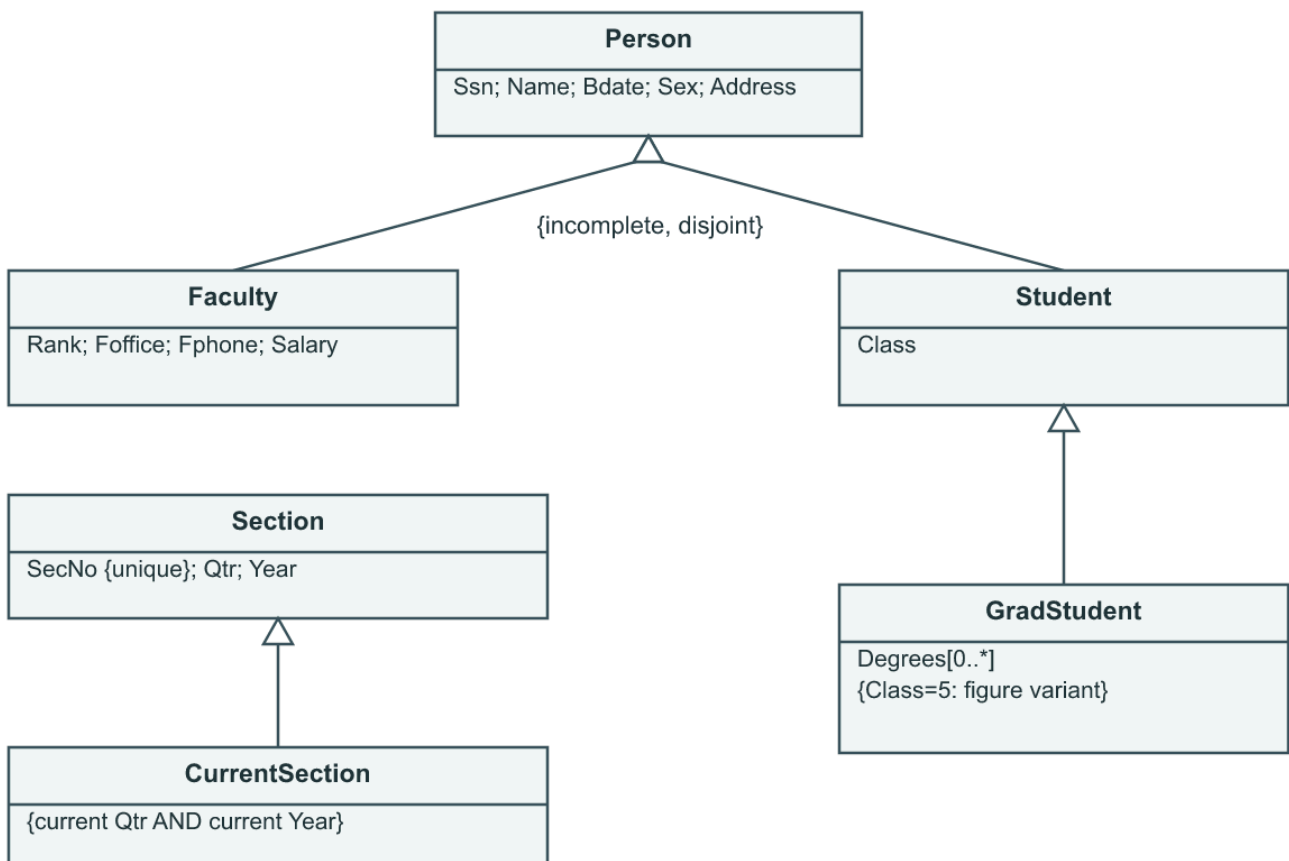


Figure 4.9 uses Class=5. The prose variant adds Class=6; do not confuse this with Figure 4.7.

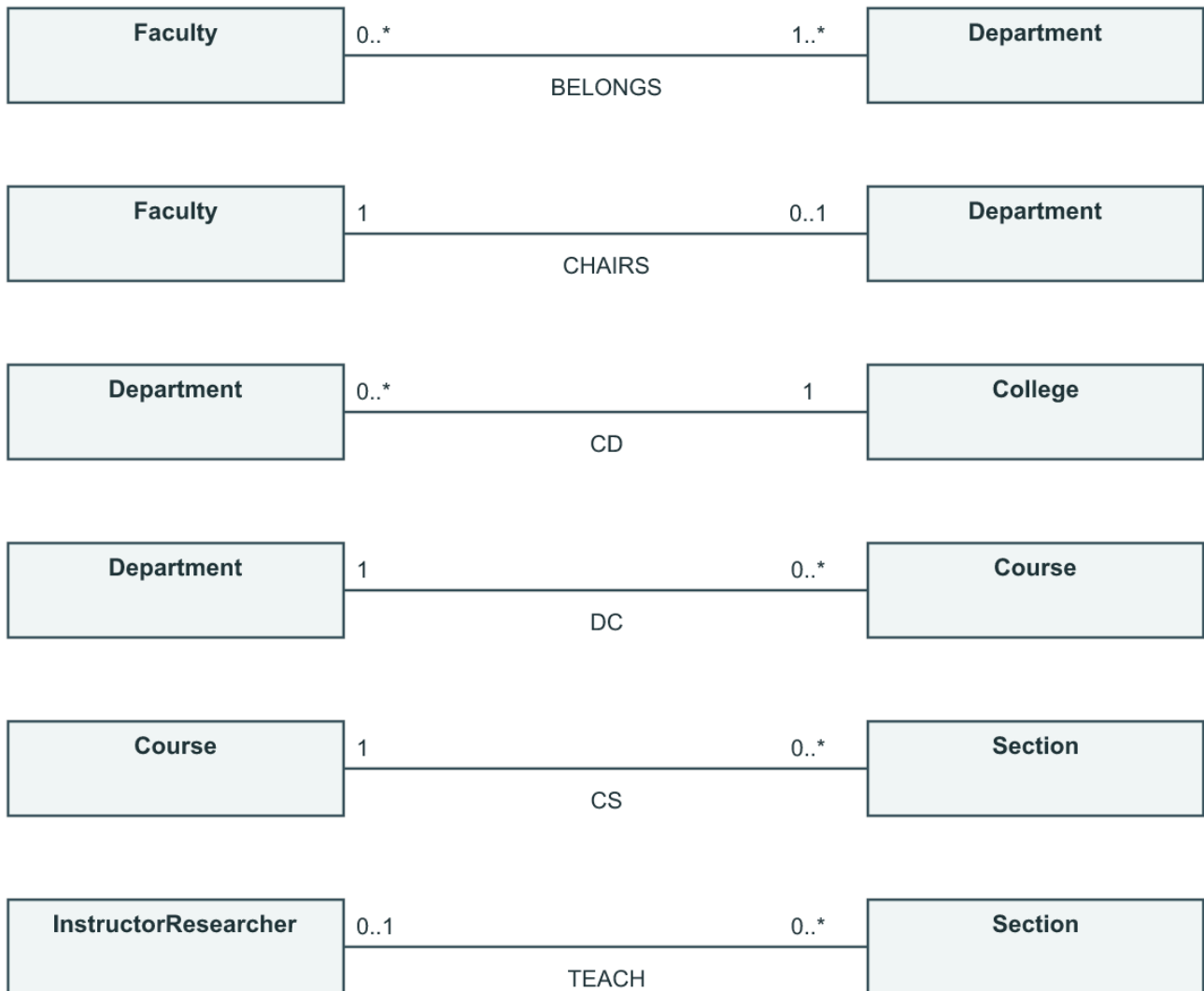
Class	Key	Properties / structured values
Person	Ssn	Name(Fname,Minit,Lname), Bdate, Sex, Address(No,Street,Apt_no,City,State,Zip)
Faculty	Inherits Ssn	Rank, Foffice, Fphone, Salary
Student	Inherits Ssn	Class
GradStudent	Inherits Ssn	Degrees[0..*], each with College, Degree, Year
InstructorResearcher	Selective identity from Faculty or GradStudent	Category mapping exempt; all Faculty plus supported graduate students
Department	Dname	Dphone, Office
College	Cname	Coffice, Dean
Course	CNo, corresponding to C#	Cname, Cdesc
Section	SecNo, corresponding to Sec#; globally unique here	Year, Qtr
CurrentSection	Inherits SecNo	Predicate on Year and Qtr
Grant	No	Title, Agency, St_date

PERSON to FACULTY/STUDENT is disjoint and partial **in this particular figure**. Do not replace it with the overlapping PERSON model of Figure 4.7.

Source conflict: Figure 4.9 prints `Class=5` for GRAD_STUDENT, whereas p.154 explicitly says `Class=5 OR Class=6`. The main UML translation here preserves the figure's `Class=5`, because the exercise asks to translate that figure. Under the prose variant, change that predicate to include 6 and leave all other links unchanged. Neither variant is silently presented as both.

UNIVERSITY: organization and teaching

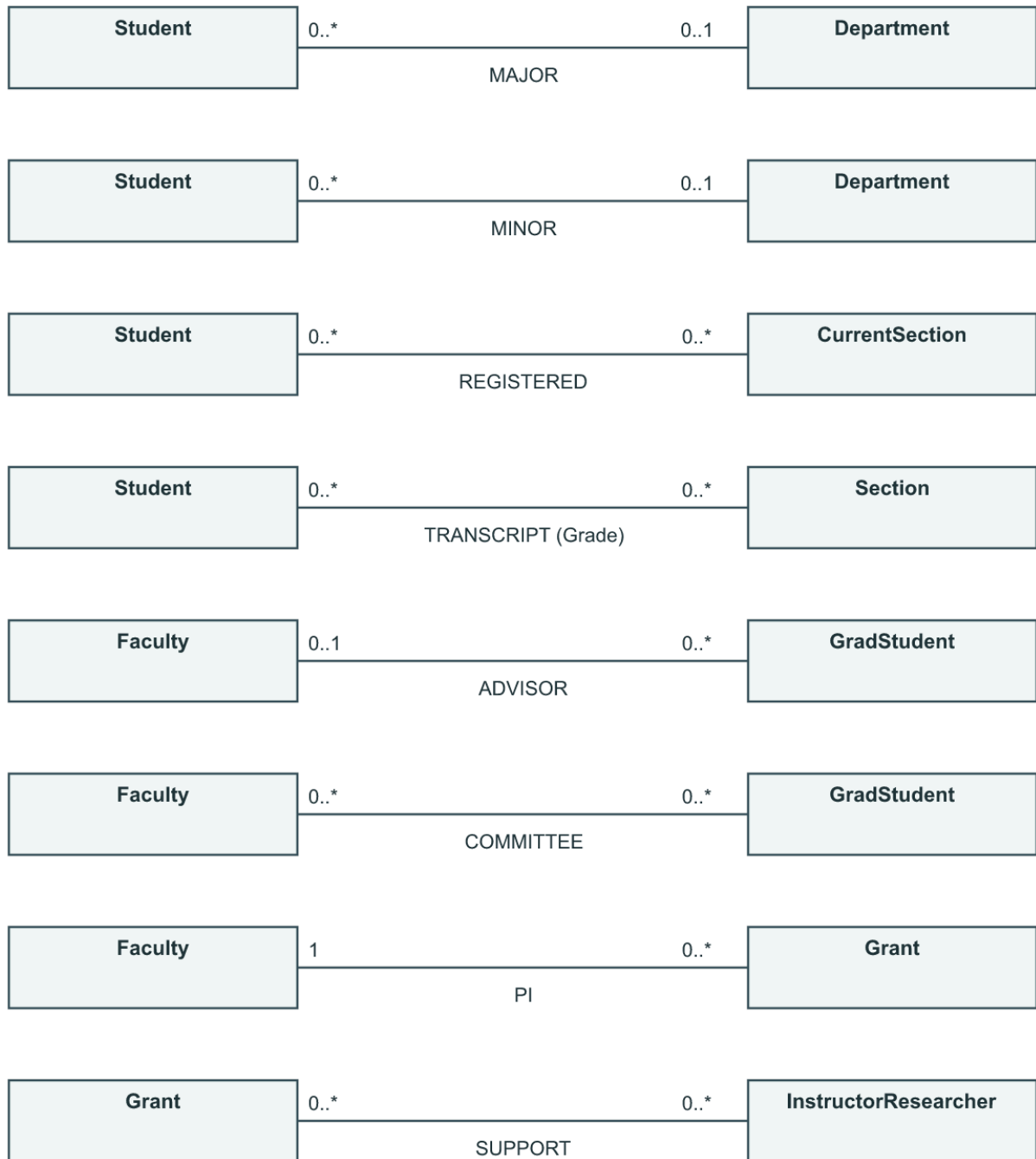
UML end multiplicities



The answer table states where narrative minima refine the figure's single lines.

UNIVERSITY: students, research and support

UML end multiplicities



Support association class: Start, End and Time. Transcript association class: Grade.

Association	Objects reached from one left object	Objects reached from one right object	Relationship attributes
Faculty-BELONGS-Department	1..* departments	0..* faculty	None
Faculty-CHAIRS-Department	0..1 department	1 faculty	None
Department-CD-College	1 college	0..* departments	None
Department-DC-Course	0..* courses	1 department	None
Course-CS-Section	0..* sections	1 course	None
InstructorResearcher-TEACH-Section	0..* sections	0..1 instructor-researcher	None
Student-MAJOR-Department	0..1 department	0..* students	None
Student-MINOR-Department	0..1 department	0..* students	None
Student-REGISTERED-CurrentSection	0..* current sections	0..* students	None
Student-TRANSCRIPT-Section	0..* sections	0..* students	Grade
Faculty-ADVISOR-GradStudent	0..* graduate students	0..1 advisor	None
Faculty-COMMITTEE-GradStudent	0..* graduate students	0..* faculty	None
Faculty-PI-Grant	0..* grants	1 investigator	None
Grant-SUPPORT-InstructorResearcher	0..* researchers	0..* grants	Start, End optional, Time

The table contains fourteen associations, including MAJOR/MINOR and REGISTERED/TRANSCRIPT separately. The complete translation uses all fourteen; the drawing panels are deliberately split for readability. Use Transcript and Support association classes for Grade and Start/End/Time. Degrees can be a multivalued structured property, rather than an independently identified College enrollment. CurrentSection is exactly the sections with Qtr=Current_qtr AND Year=Current_year, not a permanent flag that stays true.

Minima in this solution follow the narrative when it explicitly specifies a required relationship: each faculty belongs to at least one department, each department has one chair, and each grant has one PI. Other unspecified minima are zero. These are stated narrative-based refinements where Figure 4.9 itself only shows maximum ratios with single lines. Its disjointness and predicates are not reinterpreted. A literal minimum-only reading of those single lines would allow zero affiliations, chairs and PIs, contrary to that narrative.

Check	Figure translation	Prose variant
Student S1 has Class=5	GradStudent	GradStudent
Student S2 has Class=6	Not GradStudent	GradStudent
Section X is in the current year but another quarter	Not CurrentSection	Not CurrentSection

4.23. Classifying the Fifteen Pairs

Source: photographed exercise 4.23, p.171.

Choose one column for each row, interpreting the names as ordinary concepts in this exercise. An association between two roles does not assert that one role is a subtype of the other.

Row	Left	Right	Choice	Why
1	MOTHER	PERSON	c: specialization	Every mother is a person.
2	DAUGHTER	MOTHER	a: relationship	A daughter has a mother; being a daughter does not imply being a mother.
3	STUDENT	PERSON	c: specialization	Every student represented here is a person.
4	STUDENT	Student_id	b: attribute	Student_id identifies a student.
5	SCHOOL	STUDENT	a: relationship	A student attends a school.
6	SCHOOL	CLASS_ROOM	a: relationship	A classroom belongs to or is used by a school.
7	ANIMAL	HORSE	d: generalization	Every horse is an animal, not every animal a horse.
8	HORSE	Breed	b: attribute	Breed is a property under the given representation.
9	HORSE	Age	b: attribute	Age is a property, potentially derived.
10	EMPLOYEE	SSN	b: attribute	SSN is an employee identifier in the example.
11	FURNITURE	CHAIR	d: generalization	A chair is a kind of furniture.
12	CHAIR	Weight	b: attribute	Weight describes a chair.
13	HUMAN	WOMAN	d: generalization	A woman is a human.
14	SOLDIER	PERSON	c: specialization	A soldier is a person.
15	ENEMY_COMBATANT	PERSON	c: specialization	The modeled role is a kind of person, not a legal determination.

For row 2, one person may be both a daughter and a mother, but overlap does not establish subset inclusion. Breed could become an entity in a richer application; the answer follows the attribute interpretation requested by this table.

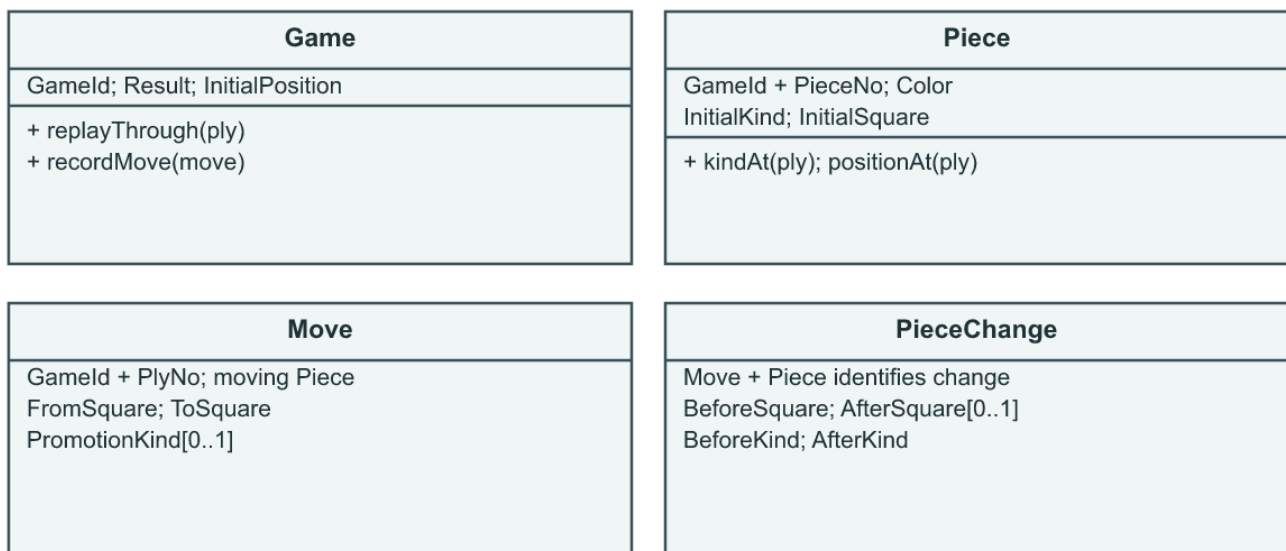
4.24. A Played Chess Game in UML

Source: photographed exercise 4.24, pp.171-172.

Store player identities independently of games. A GameSeat assigns one player to one color in a game. A Piece is a particular game piece with stable identity, not merely a piece kind. Store every move in order and retain captured pieces as historical objects. Removing a piece from the board must not delete history.

Chess UML: persistent objects and operations

UML class compartments

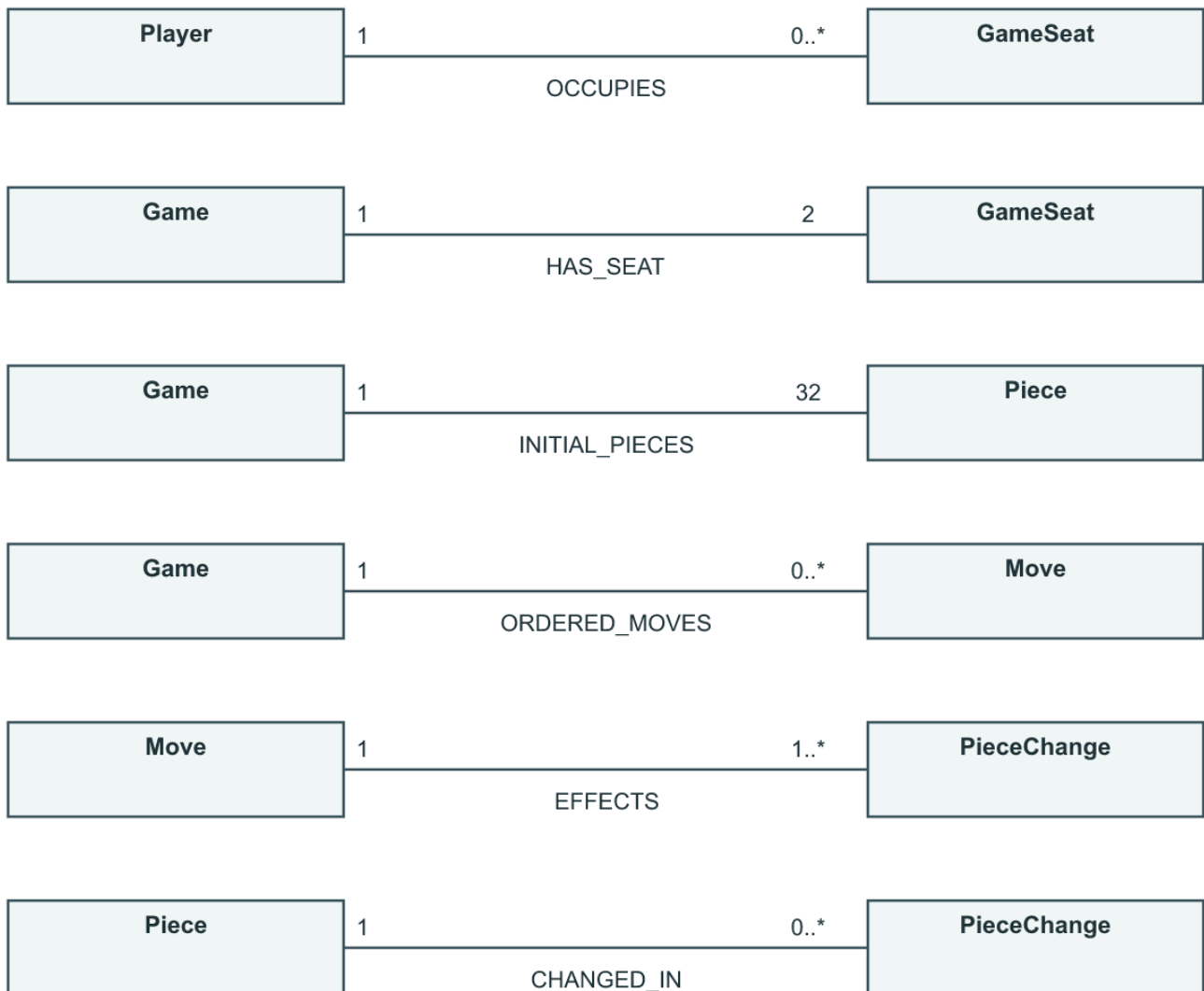


Class	Key	Properties and responsibilities
Player	PlayerId	Name
Game	Gameld	Result; initial position; replayThrough(ply), recordMove(move)
GameSeat	Gameld + Color	Color in {White,Black}; linked Player
Piece	Gameld + PieceNo	InitialKind, InitialSquare, Color; kindAt(ply), positionAt(ply)
Move	Gameld + PlyNo	FromSquare, ToSquare, PromotionKind optional; moving Piece
PieceChange	Gameld + PlyNo + PieceNo	BeforeSquare, AfterSquare optional, BeforeKind, AfterKind
Square, value type	File + Rank	File in a..h; Rank in 1..8

Each Game has exactly two GameSeats, one of each color and two distinct players. One Player may have many GameSeats in different games. Each Game has 32 initial Pieces and 0..* Moves; each Move belongs to one Game and moves one Piece from that game. Each Move has 1..* PieceChanges, each for exactly one Piece in the same game. Each Piece may occur in 0..* changes. Treat PlyNo as one player's turn, not a pair of turns; order by Gameld, PlyNo. GameSeat, Piece and Move have owner-scoped identifiers even if a UML tool does not draw double rectangles.

Chess UML: games, players and move effects

UML end multiplicities



Each Move also references one moving Piece. Same-game rules and identifiers are in the answer.

Assume the standard starting position: per color, one king, one queen, two rooks, two bishops, two knights and eight pawns. Initial squares must be distinct. PieceKind is an attribute whose value can change through promotion, not an immutable subtype that erases the original pawn's identity.

Illustrative event	PieceChange records	Result
White pawn moves a2 to a4 on an empty board path	WPa: a2 -> a4, pawn -> pawn	a2 becomes empty; WPa is at a4.
White pawn captures an opposing piece on b8 and promotes	WPa: a7 -> b8, pawn -> queen; captured black piece: b8 -> absent	WPa keeps its PieceNo; captured piece leaves the board.
Promotion requested to king	Rejected by the promotion-kind rule	Allowed choices are queen, rook, bishop, knight.

These are separate illustrative positions, not consecutive moves of one game. Promotion requires a pawn reaching its last rank. Before/after facts must agree with the state immediately before the move. Reject moving a captured piece, capturing a same-color piece, or leaving two pieces on one square.

PieceChange also permits the two moving pieces of castling or a captured piece on another square during en passant. The question excludes a full legal-move engine; representing those effects does not prove their legality. No chess engine or full game implementation is delivered. Move recording, replay, occupancy and promotion checks are operations of Game/Move; Piece supplies identity and type information. This division avoids assigning all behavior to one database attribute.

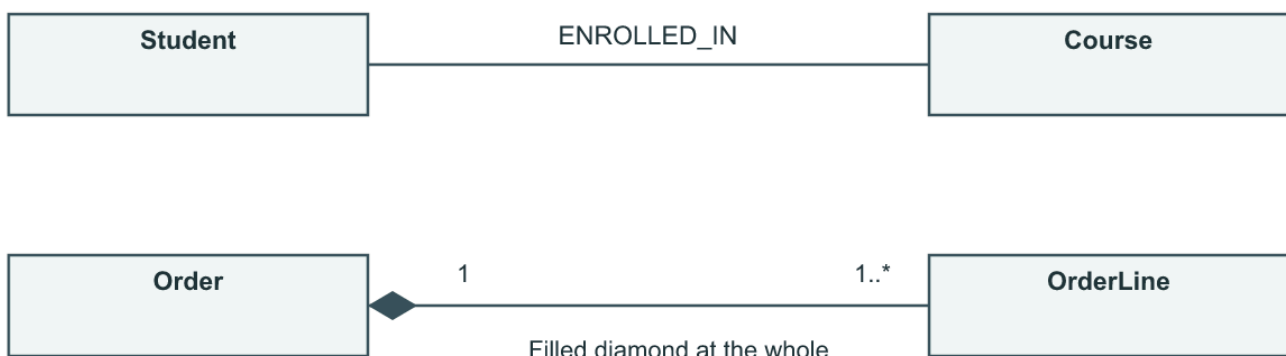
4.25. Association, Aggregation and Composition

Source: photographed exercise 4.25, p.172; 4.7.4, pp.161-163. Terminology check: [OMG UML 2.5.1](#), 9.5.3, printed p.112.

The photographed question attributes strong ownership and coincident lifetime to **association**. That premise needs correction: in UML those are **composition** (composite aggregation) semantics, not ordinary association. This is also not the other textbook copy's 4.25 chess EER question.

Ordinary association is not composition

UML



Deleting ORDER removes its remaining ORDER_LINE objects, not their referenced PRODUCT objects.

Relationship	Example	What deletion means
Ordinary association	STUDENT enrolled in COURSE	Removing enrollment does not delete student or course.
Shared aggregation	A grouping whose part-sharing rules are application-defined	Do not assume cascading deletion just from a hollow diamond.
Composition	ORDER owns ORDER_LINE	Under this model, deleting an order deletes its remaining order lines.

Use a filled diamond at the ORDER end in UML. Each ORDER_LINE belongs to at most one composite at a time; in this example it belongs to exactly one ORDER. The PRODUCT referenced by a line is not itself owned by the order and survives order deletion. For EER, use an owner-dependent ORDER_LINE with local LineNo and an identifying relationship, plus the stated existence/deletion rule.

Coincident lifetime should not be exaggerated into a claim of identical creation times. UML does not prescribe the exact creation order. A part can, where allowed, be detached before its composite is deleted. This example disallows detached ORDER_LINES, which is an explicit application constraint.

4.26. Which Drawings Are Incorrect?

Source: photographed exercise 4.26, pp.172-173; 4.3.1. The photographed (a) contains a **binary** E2-R-E3 relationship, not the ternary relationship appearing in the other copy.

Test the photographed diagrams, not another edition's version

a: valid, total + overlapping E={a,b,c}; E1={a,b}; E2={b,c} E3={x,y} R={(c,x),(c,y)}: binary E2-E3	b: valid, partial + disjoint E={a,b,c}; E1={a}; E2={b} R={(a,b)} Different objects may be related
c: superclass missing E1 and E3 link to an o circle No parent class is connected M:N relationship is not a parent	c: repair Add named superclass E Declare total or partial coverage Keep E1-E3 M:N relationship

Part	Decision	Explanation
a	Structurally valid	E1 and E2 are overlapping, total subclasses of E. E2 can participate in a binary 1:N relationship with E3. No rule forbids a subclass from having its own relationship.
b	Structurally valid	E1 and E2 are disjoint, partial subclasses of E. An ordinary 1:1 relationship may associate an E1 object with a different E2 object. Disjointness forbids shared membership, not associations.
c	Incomplete / invalid specialization notation	The o circle joins E1 and E3 as subclasses but has no superclass. The M:N relationship below it does not supply one.

For (a), take $E=\{a,b,c\}$, $E1=\{a,b\}$, $E2=\{b,c\}$, $E3=\{x,y\}$, and $R=\{(c,x),(c,y)\}$. Every E entity is covered; b overlaps; each E3 object has at most one E2 partner, while c has two E3 partners. This satisfies the printed 1 at E2 and N at E3. It would be wrong to read that 1 as only one E3 per E2.

For (b), take $E=\{a,b,c\}$, $E1=\{a\}$, $E2=\{b\}$, $R=\{(a,b)\}$. The objects a and b are different, so the relationship does not violate disjointness. c demonstrates partial coverage. If an additional business rule required each R link to connect an object to itself, nonempty R would conflict with disjointness; that extra rule is not in the figure.

For (c), add a named superclass E above the circle, connect it, and declare total or partial coverage and identifiers. Then E1/E3 may overlap and still have an M:N relationship. Replacing the circle with a union category is not equivalent. Unnamed attribute ovals in these schematic questions are treated as placeholders; a usable final schema must name its attributes and keys.

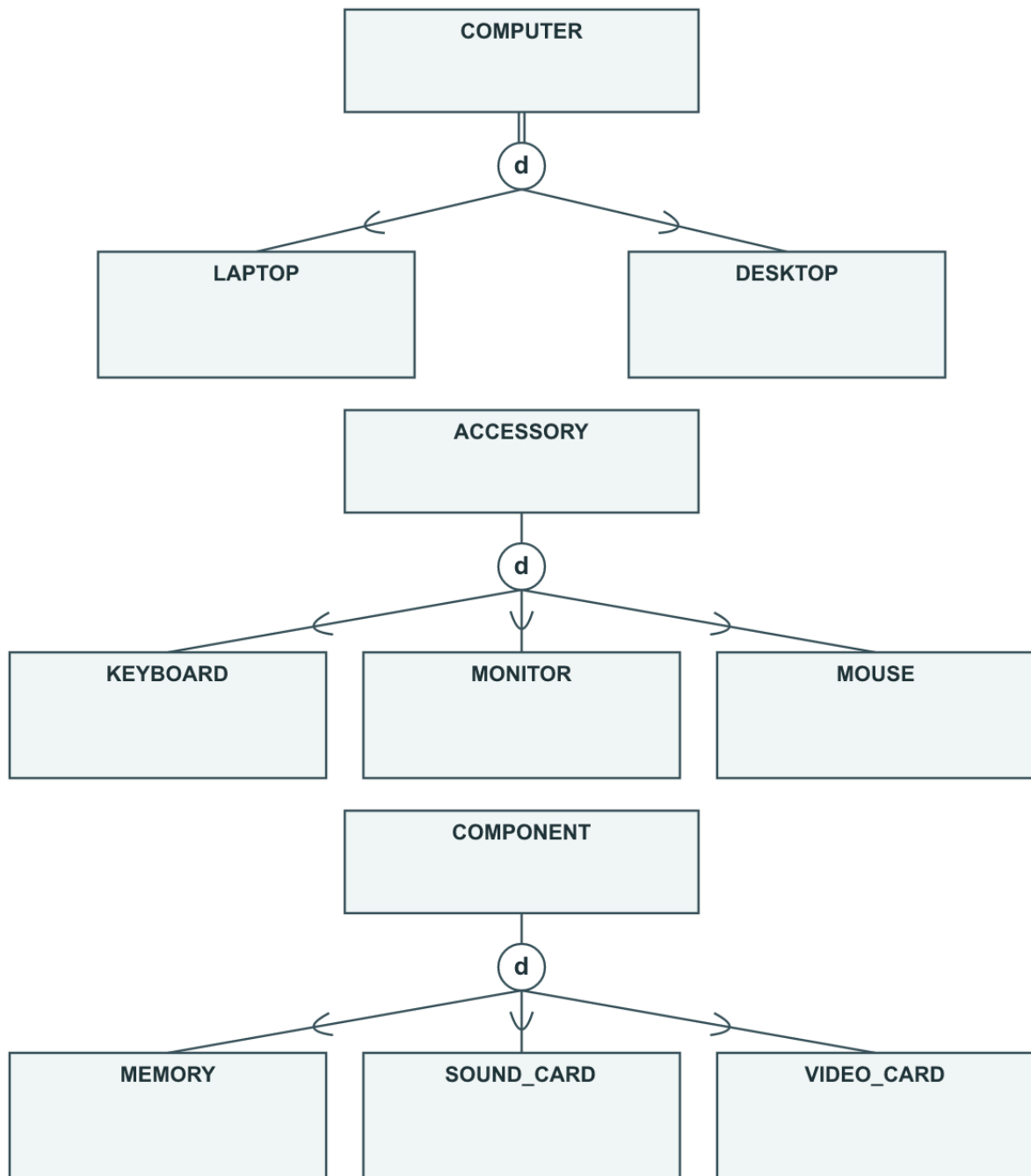
4.27. Computer Systems: Keys, Counts and Narrative

Source: photographed exercise 4.27, p.173.

Interpret COMPUTER, ACCESSORY and COMPONENT as **product/configuration types**, not individual serial-numbered units. This assumption allows the same memory option or mouse model to be compatible with several computer models. Software identity includes version through a separate identifier. Maximum counts below are proposed design choices; the drawn participation and specialization constraints are preserved.

Computer product classes: preserve partial versus total

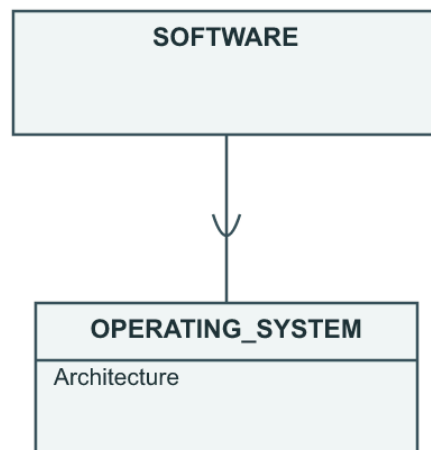
EER



OPERATING_SYSTEM is also a partial subclass of SOFTWARE; its properties appear in the table.

Every operating system is software

EER



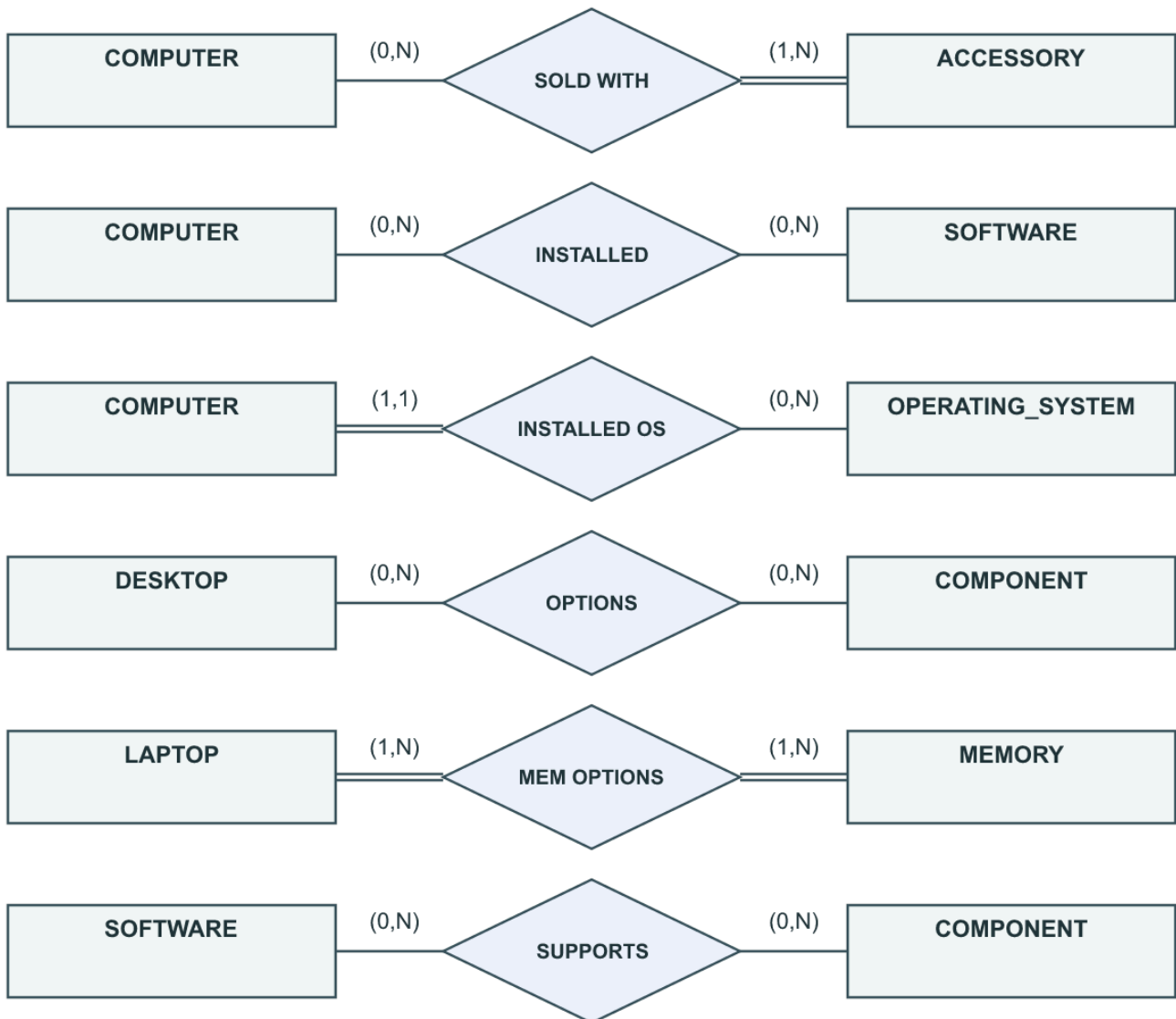
SOFTWARE also includes non-operating-system products. SUPPORTS can involve those products too.

Type	Key	Proposed local attributes
COMPUTER	ComputerModelId	Manufacturer, ModelName, BasePrice
LAPTOP	Inherits ComputerModelId	ScreenSizeInches, BatteryWh
DESKTOP	Inherits ComputerModelId	CaseFormFactor
ACCESSORY	AccessoryModelId	Name, Manufacturer, Price
KEYBOARD	Inherits AccessoryModelId	Layout
MONITOR	Inherits AccessoryModelId	Resolution
MOUSE	Inherits AccessoryModelId	ButtonCount
COMPONENT	ComponentModelId	Name, Price
MEMORY	Inherits ComponentModelId	CapacityGB, MemoryStandard
SOUND_CARD	Inherits ComponentModelId	ChannelCount
VIDEO_CARD	Inherits ComponentModelId	VideoMemoryGB
SOFTWARE	SoftwareId	ProductName, Version
OPERATING_SYSTEM	Inherits SoftwareId	Architecture

COMPUTER is totally and disjointly specialized into LAPTOP and DESKTOP. ACCESSORY and COMPONENT have disjoint but **partial** displayed subtypes; their superclass-to-circle lines are single. OPERATING_SYSTEM is a subclass of SOFTWARE, not a separate unrelated class.

Computer-system relationships with proposed maxima

ER min-max participation



ACCESSORY and both MEM_OPTIONS participants have minimum one in the photographed figure.

Relationship	First participant min-max	Second participant min-max	Why these maxima?
COMPUTER-SOLD_WITH-ACCESSORY	COMPUTER (0,N)	ACCESSORY (1,N)	A model can include many accessories; an accessory model can be bundled with many models.
COMPUTER-INSTALLED-SOFTWARE	COMPUTER (0,N)	SOFTWARE (0,N)	Many installed software products and many computers per product
COMPUTER-INSTALLED_OS-OPERATING_SYSTEM	COMPUTER (1,1)	OPERATING_SYSTEM (0,N)	One configured OS per computer model in this answer; OS reused across models
DESKTOP-OPTIONS-COMPONENT	DESKTOP (0,N)	COMPONENT (0,N)	Many configurable components; components reused across desktop models
LAPTOP-MEM_OPTIONS-MEMORY	LAPTOP (1,N)	MEMORY (1,N)	Every laptop and every represented memory option participates, as the double lines require.
SOFTWARE-SUPPORTS-COMPONENT	SOFTWARE (0,N)	COMPONENT (0,N)	Software supports multiple components; a component can be supported by multiple products.

Narrative: the catalog contains only laptop and desktop computer models. It records bundled accessories, installed software, and one configured operating system per model. Some accessories are keyboards, monitors or mice; the partial classification also permits other accessories. Some components are memory, sound cards or video cards, with other components permitted. Desktop models can offer arbitrary component options. Every laptop has at least one memory option, and every memory option represented here is offered for at least one laptop. The SUPPORTS relationship connects **SOFTWARE to COMPONENT**, not just OPERATING_SYSTEM to COMPONENT.

A configured OS is software, but the separate INSTALLED_OS and INSTALLED lines do not themselves guarantee that the same installation appears on both. Add INSTALLED_OS is a subset of INSTALLED if INSTALLED is intended as the complete software inventory; this answer adopts that explicitly. It does not assume every supported component is actually installed. Allowing multi-boot would change COMPUTER's INSTALLED_OS maximum to N, a different declared design.

Proposed record	Result under this answer
Laptop L1 offers memory M1; desktop D1 offers the same M1	Allowed; MEMORY is a COMPONENT.
Memory M2 exists but is not an option for any laptop	Rejected by the figure's total MEM_OPTIONS participation.
Software S1 is not an OS but supports video card V1	Allowed; SUPPORTS starts at SOFTWARE.
A computer model belongs to neither LAPTOP nor DESKTOP	Rejected by total specialization.

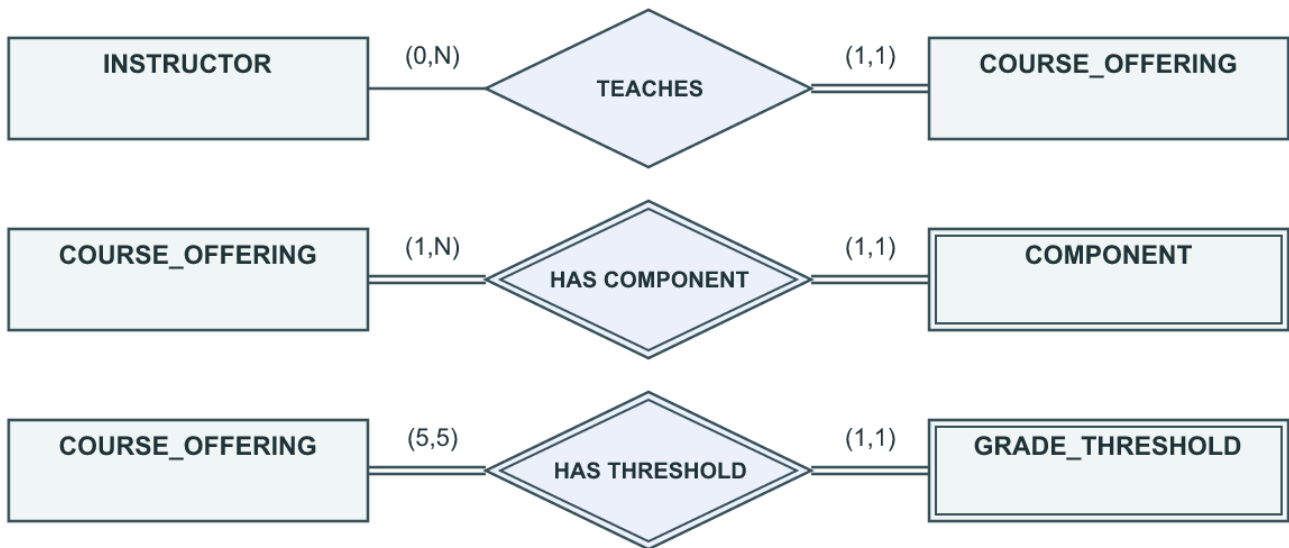
4.28. GRADE_BOOK

Source: photographed laboratory exercise 4.28, pp.173-174. Completion: conceptual design and calculation supplied; modeling-application work not completed.

Represent the question's course as a particular offering, identified by (CourseNumber, SectionNumber, Term). A grading component belongs to exactly one offering. A score belongs to a student and a component of an offering in which that student is enrolled, not merely to the student or course number.

Each course offering has its own grade rules

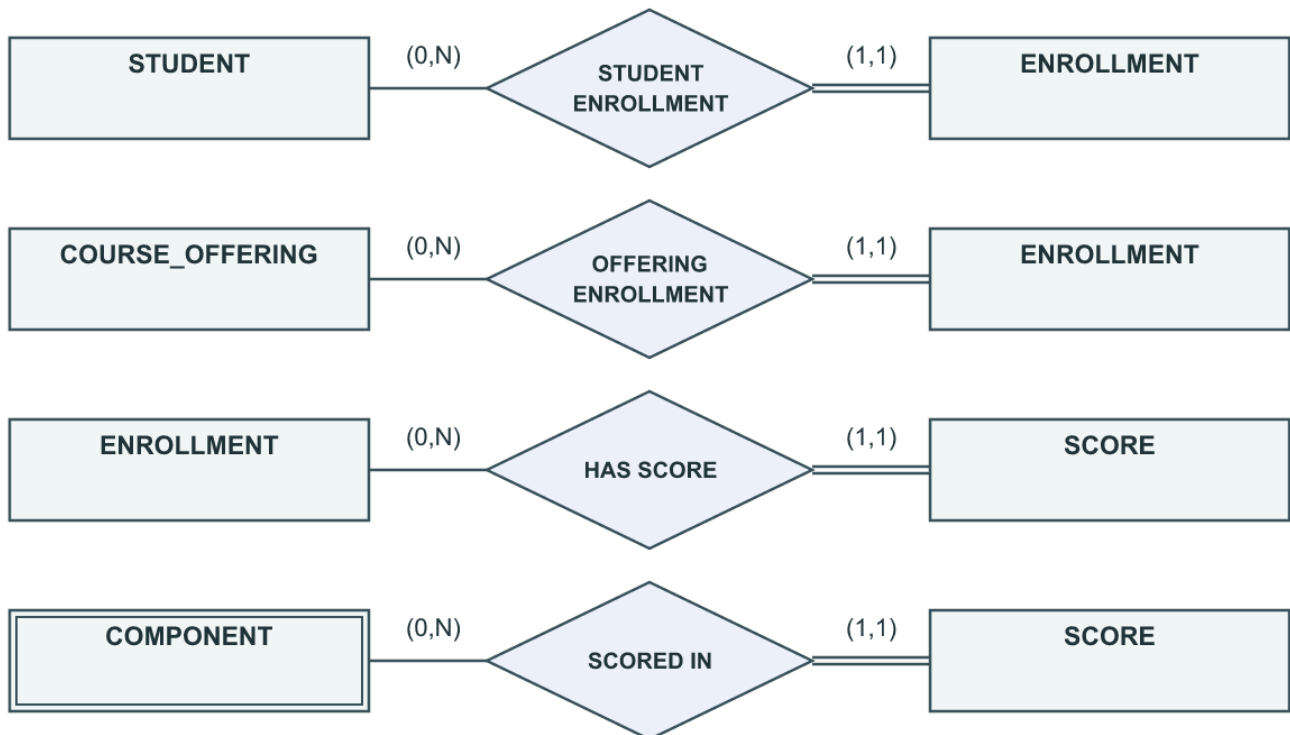
ER min-max participation



Type	Identifier	Attributes
STUDENT	StudentId	FirstName, LastName, Email
INSTRUCTOR	InstructorId, added identifier	Name
COURSE_OFFERING	CourseNumber + SectionNumber + Term	No extra attribute required
COMPONENT, weak	Offering key + ComponentNo	Name, MaxPoints, WeightPercent
GRADE_THRESHOLD, weak	Offering key + Letter	MinimumPoints; Letter in A,B,C,D,F
ENROLLMENT	StudentId + Offering key	Optional enrollment status
SCORE	StudentId + Component's full key	PointsEarned

One score for one enrollment and one component

ER min-max participation



SCORE requires matching offering keys and unique enrollment/component identity.

Assume one instructor teaches an offering; an instructor can teach zero or many offerings. Every offering has one or more components. Each offering has exactly five thresholds, one per specified letter; thresholds are not global because instructors set them for each offering. An offering and a student each have zero or more enrollments. Each SCORE connects exactly one ENROLLMENT and one COMPONENT **from the same offering**. Each enrollment/component pair has at most one current score. A missing score is not automatically zero.

Component	MaxPoints	WeightPercent	PointsEarned	Weighted contribution
Midterm	100	20	84	16.8
Final	100	50	80	40.0
Project	50	30	45	27.0
Total	Not summed as the denominator	100	Raw sum is not the course score	83.8

For this example only, weights sum to 100 and the total is on a 100-point scale. Each contribution is $(\text{PointsEarned} / \text{MaxPoints}) * \text{WeightPercent}$. With thresholds A=90, B=80, C=70, D=60, F=0, the highest satisfied threshold for 83.8 is B. The sample extends the textbook's midterm calculation; it does not set this course's assessment policy.

Require $\text{MaxPoints} > 0$, nonnegative weights and, for the no-extra-credit example, $0 \leq \text{PointsEarned} \leq \text{MaxPoints}$. Declare any extra-credit policy separately. The word 'usually' in the question does not justify an unconditional 100-sum rule for every possible offering. Do not normalize or assign a letter when weights, missing scores or threshold ordering have not been resolved. A duplicate score or a score for a component in another offering is rejected.

To finish the lab in a modeling tool, enter the owner and association structures, the composite identifiers and the same-offering constraint. Test one valid score, an unregistered student, a duplicate component score and a component from the wrong offering. Export the diagram and retain the native model file; those tool-specific artifacts are not supplied here.

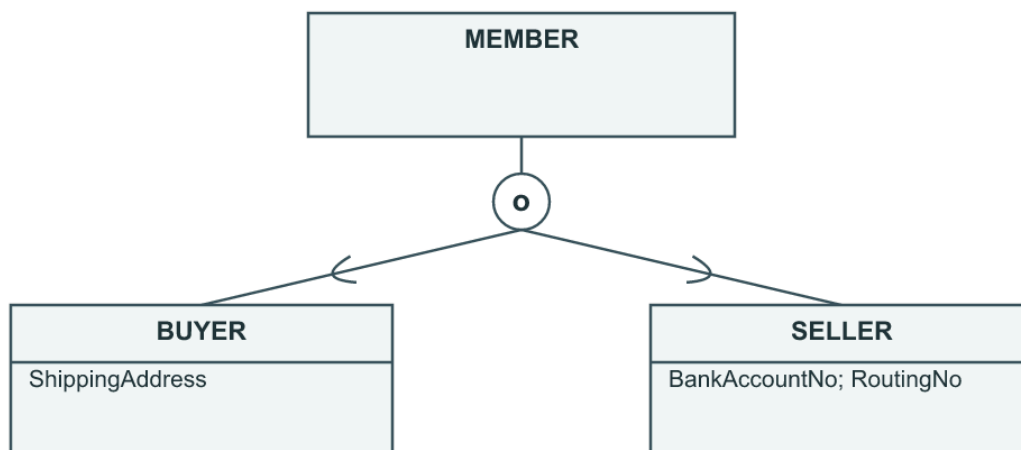
4.29. ONLINE_AUCTION

Source: photographed laboratory exercise 4.29, pp.174-175. Completion: conceptual design and example checks supplied; modeling-application work not completed.

Use an overlapping, partial specialization MEMBER to BUYER and SELLER. A member can hold both roles; a newly registered member may have neither under this explicit assumption. Separate bids, the resulting transaction, and feedback: a winning bid is not proof that a sale was completed.

One member can be both buyer and seller

EER



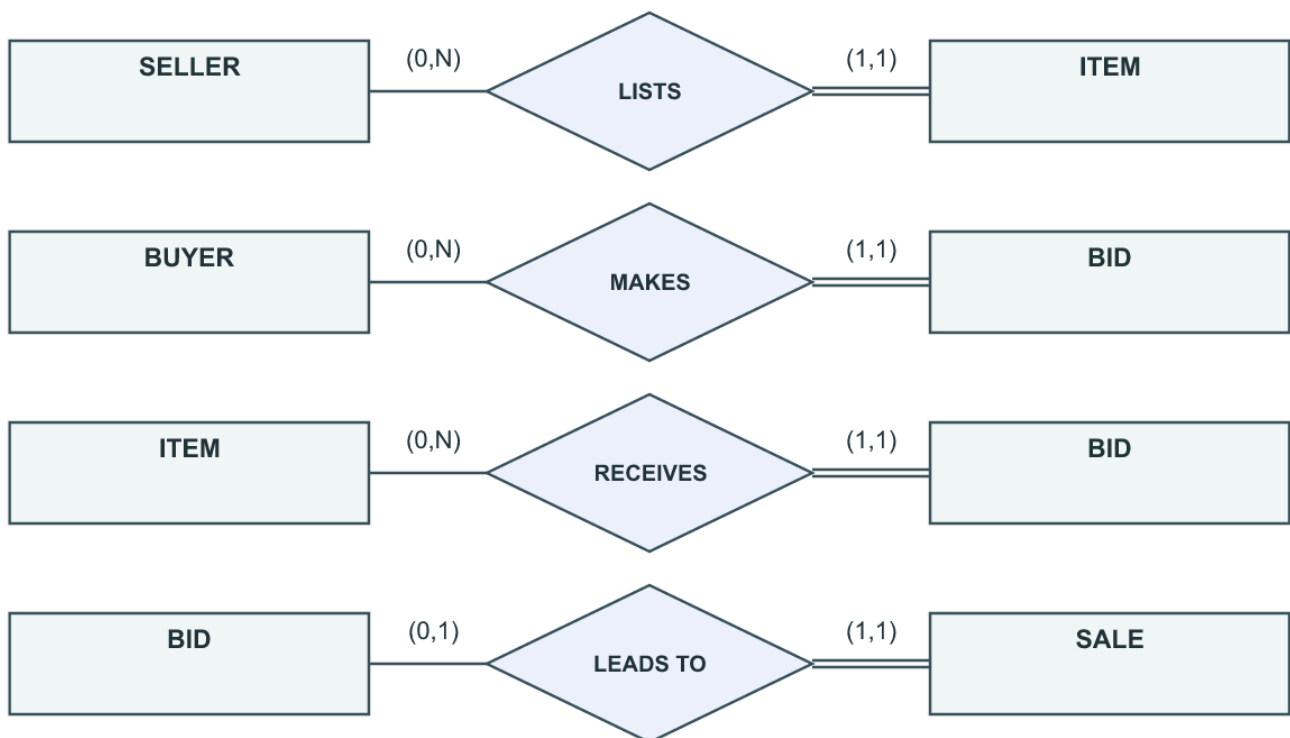
Partial membership permits new accounts without a buying or selling role in this answer.

Type	Key	Attributes
MEMBER	MemberNo	Email, Name, Password, HomeAddress, Phone
BUYER	Inherits MemberNo	ShippingAddress
SELLER	Inherits MemberNo	BankAccountNo, RoutingNo
ITEM	ItemNo	Title, Description, StartingPrice, BidIncrement, StartTime, EndTime
CATEGORY	CategoryId	Name
BID	BidId, added unique identifier	Price, BidTime
SALE	SaleId, added identifier	Status, CompletedTime optional
FEEDBACK	SaleId + AuthorMemberNo	Rating, Comment

Password is the question's credential requirement, not an instruction to store raw passwords. No passwords, real bank data, routing numbers or actual member records are included in these examples. Their storage and access controls require an implementation design beyond this conceptual answer.

A bid is not a completed sale

ER min-max participation



Only the winner can lead to a sale; the sale may still be incomplete.

Relationship	First participant	Second participant
SELLER-LISTS-ITEM	SELLER (0,N)	ITEM (1,1)
CATEGORY-CONTAINS-ITEM	CATEGORY (0,N)	ITEM (1,1), one leaf category assumed
CATEGORY-PARENT_OF-CATEGORY	Parent (0,N)	Child (0,1)
BUYER-MAKES-BID	BUYER (0,N)	BID (1,1)
ITEM-RECEIVES-BID	ITEM (0,N)	BID (1,1)
BID-LEADS_TO-SALE	BID (0,1)	SALE (1,1), winning bid only
SALE-HAS-FEEDBACK	SALE (0,2)	FEEDBACK (1,1)
MEMBER-WRITES-FEEDBACK	MEMBER (0,N)	FEEDBACK (1,1)
MEMBER-RECEIVES-FEEDBACK	MEMBER (0,N)	FEEDBACK (1,1)

CATEGORY is the auction's classification hierarchy, not an EER union category. Assume a rooted tree: one root with no parent, every other category with one parent, no cycles, and exactly one leaf category per item. The root-to-leaf path provides higher classifications. Multiple independent tags would require a different item-category cardinality.

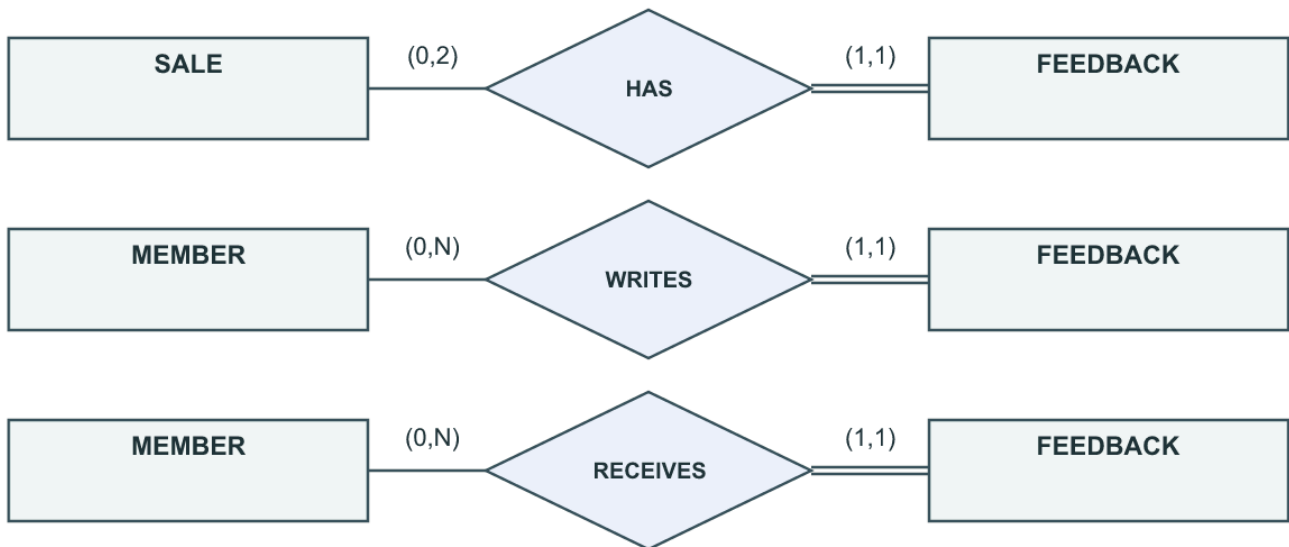
Declare a simple auction rule for the example: all bids use one currency; bids must arrive at or after StartTime and strictly before EndTime. The first bid must meet StartingPrice; each later accepted bid must be at least the current highest price plus BidIncrement. Bids are processed in a stable recorded order. This rule is an explicit assumption, not a complete real auction mechanism.

Item I1, starts at 100, increment 10	Buyer	Bid price	Result
Bid B1 during auction	M1	100	Accepted
Bid B2 later during auction	M2	110	Accepted
Bid B3 later during auction	M1	130	Accepted; M1 can bid again
Bid B4 at EndTime	M2	150	Rejected by the time rule

B3 is the winner at closure; a SALE may then be created. One item has at most one sale in this single-auction-per-item scope, and its bid must be that item's winner. Seller is obtained from the item; buyer is obtained from the bid. They must be distinct. If accepting equal prices under a different rule, declare a tie-breaker before selecting a winner. No accepted bids means no winner and no sale.

Feedback belongs to a completed sale and two parties

ER min-max participation



Author and recipient must be opposite sale parties; one feedback per sale and author.

Feedback is optional and allowed only when Status='completed'. Rating is an integer 1-10. Author and recipient must be opposite parties of this sale. The (SaleId, AuthorMemberNo) key permits one feedback record per party, at most two in total; it rejects duplicate feedback and outsider reviews. Storing only one Price on BUYER-ITEM would lose the repeated B1/B3 bid history.

For the tool lab, enter the role overlap, recursive category association, repeatable BID entity and feedback uniqueness. Validate a member in both roles, two bids by the same buyer, an item with no bids, and rejected outsider feedback. Native tool construction and export remain to be performed.

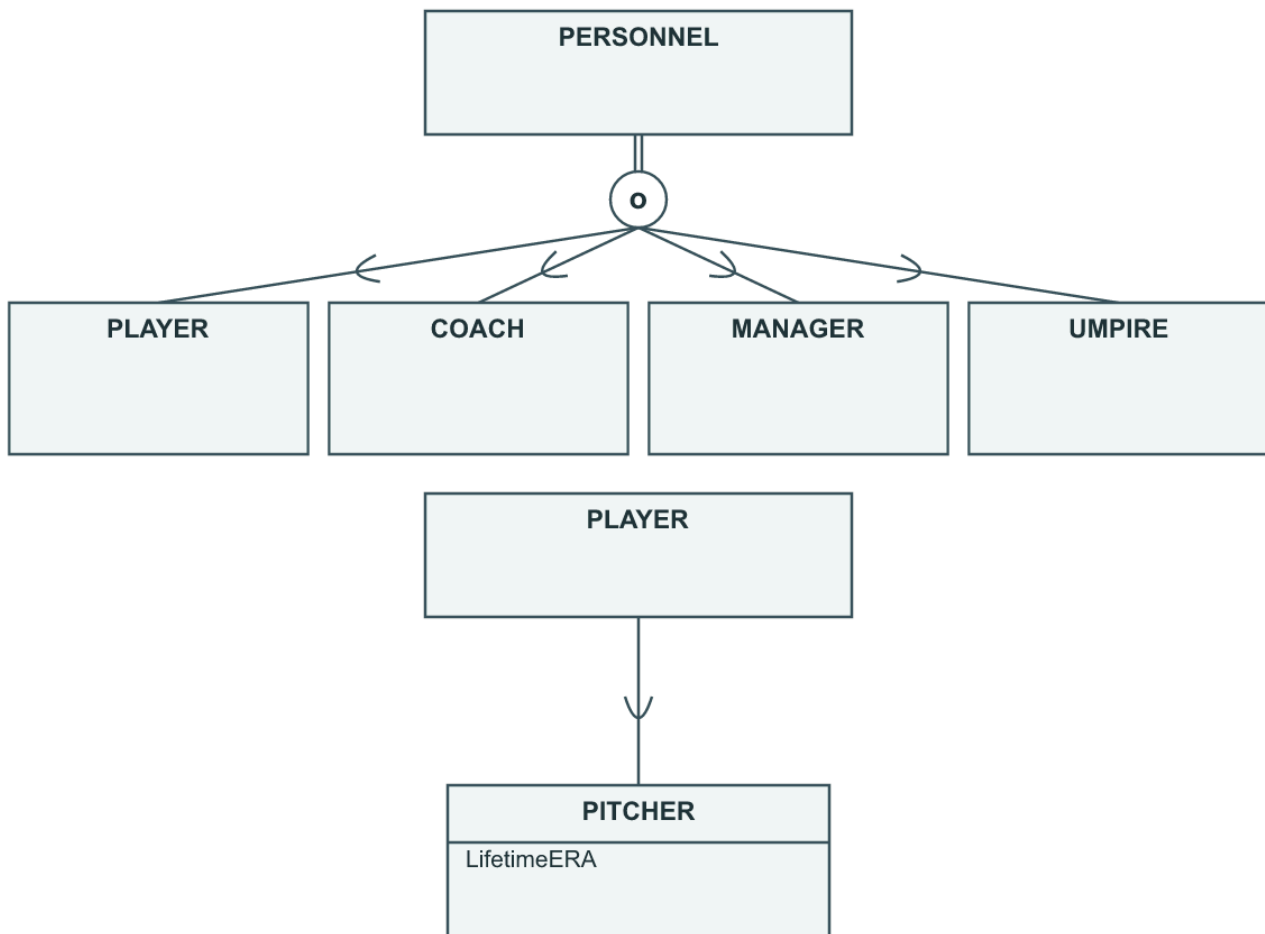
4.30. BASEBALL

Source: photographed laboratory exercise 4.30, p.175. Completion: conceptual design and score checks supplied; modeling-application work not completed.

PERSONNEL holds the shared identity and birth information. PLAYER, COACH, MANAGER and UMPIRE are overlapping, total subclasses in this answer: every represented person has at least one listed role, and the question does not forbid holding two. PITCHER is a partial subclass of PLAYER.

Baseball personnel share a common identifier

EER



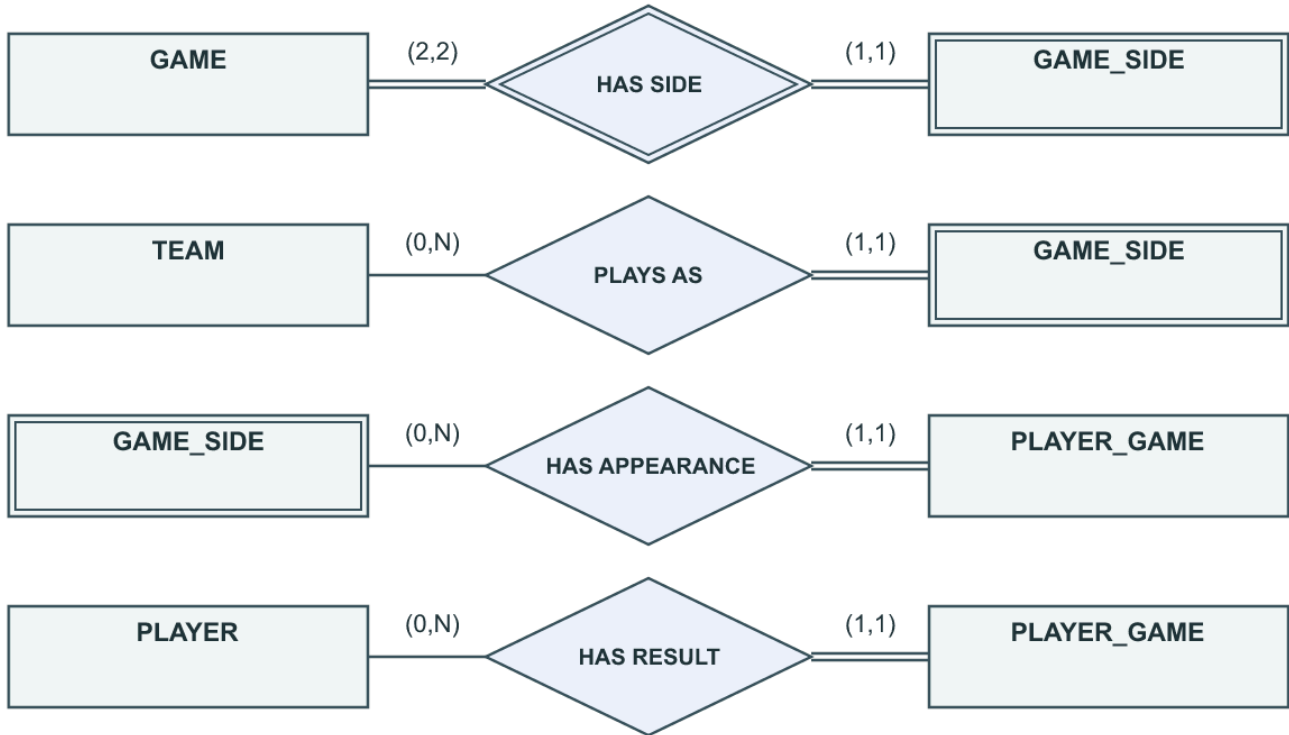
PITCHER is a partial PLAYER subclass. PLAYER adds BattingOrientation and LifetimeBA.

Type	Key	Attributes
PERSONNEL	PersonnelId	FirstName, LastName, BirthDate, BirthPlace
PLAYER	Inherits PersonnelId	BattingOrientation in left/right/switch, LifetimeBA
PITCHER	Inherits PersonnelId	LifetimeERA
COACH, MANAGER, UMPIRE	Inherit PersonnelId	No extra attributes required by the question
TEAM	Name	City, Division, League
GAME	GameId, added identifier	GameDate, Status
GAME_SIDE, weak	GameId + Side; Side home/visitor	Runs, Hits, Errors
PLAYER_GAME	GameId + PlayerId	Singles, Doubles, Triples, HomeRuns

Assume a single season with fixed rosters and staff assignments, no team transfers within the represented period, and distinct game IDs even for two games between the same teams on the same date. Division and League are attributes here; 'Central' is not assumed globally unique across leagues.

Keep team results separate from player results

ER min-max participation



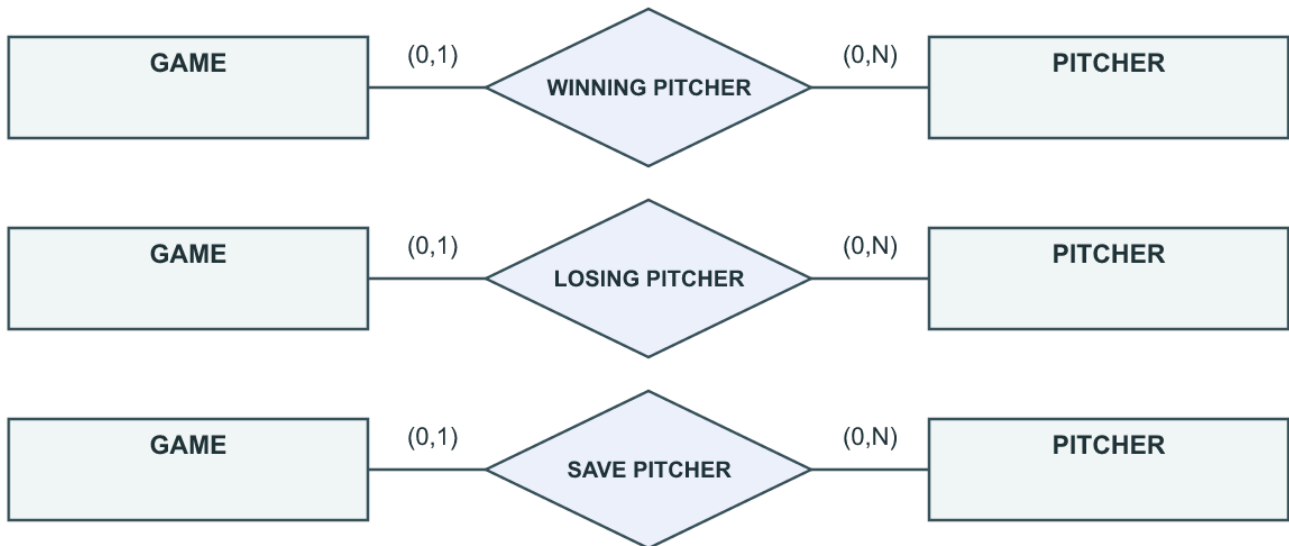
Exactly one home and one visitor side, with different teams. A player result must use that game's side.

Relationship	First participant	Second participant
MANAGER-MANAGES-TEAM	MANAGER (0,1)	TEAM (1,1)
COACH-COACHES-TEAM	COACH (0,1)	TEAM (1,N), staffed-team assumption
PLAYER-ROSTER-TEAM	PLAYER (0,1)	TEAM (1,N), active-team assumption
GAME-HAS_SIDE-GAME_SIDE	GAME (2,2)	GAME_SIDE (1,1)
TEAM-PLAYS_AS-GAME_SIDE	TEAM (0,N)	GAME_SIDE (1,1)
GAME-HAS_RESULT-PLAYER_GAME	GAME (0,N), positive when played	PLAYER_GAME (1,1)
PLAYER-HAS_RESULT-PLAYER_GAME	PLAYER (0,N)	PLAYER_GAME (1,1)
GAME_SIDE-HAS_APPEARANCE-PLAYER_GAME	GAME_SIDE (0,N)	PLAYER_GAME (1,1)
UMPIRE-OFFICIATES-GAME	UMPIRE (0,N)	GAME (0,N)

OFFICIATES is an added association to make umpire participation recordable; the question does not specify the number of umpires. GAME_SIDE's side values must be exactly {home,visitor}, not two rows both marked home. Teams must differ. Each PLAYER_GAME references one side of the same game and a player on that team's roster under the fixed-roster assumption. Retain zero-hit appearances as well as positive-hit appearances.

Pitcher decisions are conditional on game completion

ER min-max participation



Finished decisive games require a winner and loser pitcher. A save remains conditional.

Add GAME-WINNING_PITCHER-PITCHER and GAME-LOSING_PITCHER-PITCHER with GAME (0,1), PITCHER (0,N), plus GAME-SAVE_PITCHER-PITCHER with the same local bounds. A finished decisive game must have exactly one winning pitcher and one losing pitcher, with a save pitcher only when a save is awarded. These conditional minima cannot be expressed by the unconditional (0,1) alone. The winning/losing pitchers must belong to the winning/losing sides; a save pitcher belongs to the winning side and is not the winning pitcher.

Assume finished games here have a decisive result, so the side with more Runs wins. A suspended or unfinished game is not assigned final pitcher decisions. Recording award recipients is not an implementation of official award rules.

Illustrative finished game G1	Runs	Hits	Errors	Meaning
Home team T1	5	8	1	Winner because 5 > 3
Visiting team T2	3	7	0	Loser despite fewer errors

For one player with Singles=2, Doubles=1, Triples=0, HomeRuns=1, total hits are 4; total bases are 8. These are different counts. Sum player hit counts within each side to reconcile the team's Hits when all appearances are represented. All count fields must be nonnegative integers. LifetimeBA and LifetimeERA cannot be derived from these four per-game hit counts alone: the necessary at-bat, earned-run and innings information is absent. Retain them as supplied lifetime statistics, without pretending the diagram computes them.

In the modeling tool, test duplicate home sides, a player linked to the wrong game's side, a missing winning pitcher in a finished game, and a player who is also a coach. Historical roster changes would require dated memberships; they are outside the explicitly fixed-roster example. Tool files are not supplied.

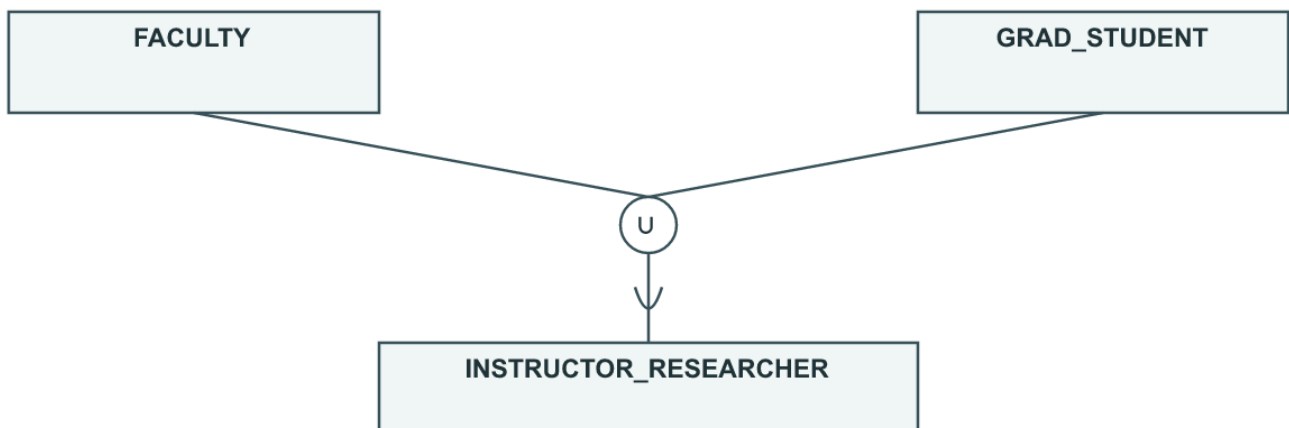
4.31. UNIVERSITY Modeling Lab

Source: photographed laboratory exercise 4.31, p.175; Figure 4.9, p.153. Completion: conceptual translation and a notation comparison plan supplied; the required tool entry and observed comparison are not completed.

Use the complete Figure 4.9 class/property inventory and all fourteen associations in Answer 4.22. Unlike the different Chapter 3 UNIVERSITY model, this schema includes grants, graduate committees and INSTRUCTOR_RESEARCHER. The following original EER diagram specifies the category. The inheritance diagram and association tables in 4.22 supply the predicates and other links.

UNIVERSITY category membership uses OR, not AND

EER



For an exact figure translation, use Class=5. If modeling the narrative, use Class=5 OR Class=6 and record the correction in the model description. The narrative also requires all faculty to be in INSTRUCTOR_RESEARCHER and includes graduate students supported by teaching or research. It does not require every graduate student to belong. Total category membership would therefore overstate the requirement.

Textbook feature	Equivalent representation to enter	What must be checked in the chosen tool
Key underline	Identifier/unique property constraint	Uniqueness is stored, not merely drawn.
d, partial PERSON specialization	Disjoint, incomplete generalization set	Faculty and Student cannot share an instance in this model.
GRAD_STUDENT predicate	Derived membership or explicit constraint	The selected Class=5 versus {5,6} variant is documented.
Composite Name and Address	Structured value types or decomposed properties	Components are not lost.
Multivalued composite Degrees	Repeated structured value or owned degree records	Each degree retains College, Degree and Year.
SUPPORT and TRANSCRIPT attributes	Association classes or associative entities	The values depend on both participants.
Category INSTRUCTOR_RESEARCHER	Native union support, or explicit membership alternatives	OR, not AND; all Faculty included, only eligible graduates included
CURRENT_SECTION predicate	Constraint on Year and Qtr	Membership changes with the selected current period.

These are **proposed equivalences**, not an observed ERwin/Rational Rose comparison. To complete the literal lab, enter the model, inspect the tool's actual symbols and constraint support, save its native project, and replace this plan with a comparison of the actual exported diagram. Test a Class=6 student to expose the source conflict, a faculty member with no category membership, and a grant without a PI under the narrative-based variant.

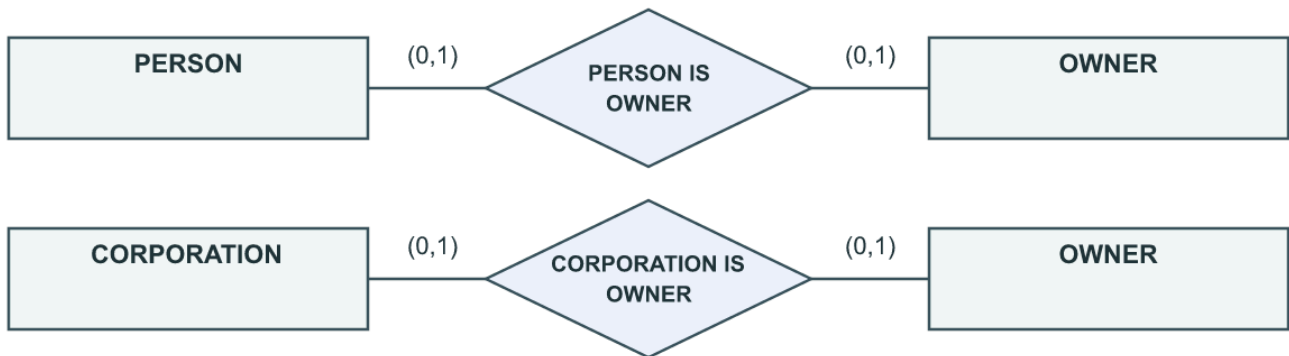
4.32. AIRPORT OWNER Without Native Category Support

Source: photographed laboratory exercise 4.32, p.175; Figure 4.12, p.170. Completion: conceptual replacement and membership checks supplied; modeling-application work not completed.

Retain the full AIRPORT design from Answer 4.21. To implement the hint, give OWNER a new surrogate OwnerId and connect it separately to PERSON and CORPORATION. This changes how identity is represented, so the extra constraints below are essential. It must not become an owner who is simultaneously both a corporation and a person.

Represent category sources with two exclusive alternatives

ER min-max participation



Additional rule: exactly one source link per OWNER, never zero or two. OwnerId is newly introduced.

Relationship	First participant	OWNER participant
PERSON-PERSON_IS_OWNER-OWNER	PERSON (0,1)	OWNER (0,1)
CORPORATION-CORPORATION_IS_OWNER-OWNER	CORPORATION (0,1)	OWNER (0,1)

For every OWNER, the number of these two source links must sum to **exactly one**. This is an exclusive-or constraint across relationships. Neither pair of local (0,1) constraints enforces it alone. The source-side maximum one prevents duplicating the same person's or corporation's owner identity. PERSON.Ssn and CORPORATION.Name keep their different source keys; OwnerId is not inferred by joining those unrelated key domains.

Owner record	Person source	Corporation source	Result
O1	P1	None	Valid person owner
O2	None	C1	Valid corporate owner
O3	None	None	Invalid: no defining source
O4	P2	C2	Invalid: two defining sources
O5	P1, already source of O1	None	Invalid: duplicate owner identity for P1

Keep AIRPLANE-OWNS-OWNER as M:N with Pdate on the pair. O1 and O2 may own the same plane; the source exclusivity does not forbid co-ownership. Preserve SERVICE's composite partial key Date/Workcode, its AIRPLANE owner, and MAINTAIN, FLIES, WORKS_ON, OF_TYPE and STORED_IN. Deleting a source entity must not leave an OWNER without a valid source; either reject deletion or perform an explicitly authorized dependent change.

For the tool lab, create the two relationships, unique source references and the cross-relationship exactly-one constraint. If the application can display but not enforce the last rule, retain it as a documented rule and test it in the eventual database/application. A visually plausible pair of lines is not evidence of enforced category semantics. No native model file is supplied.

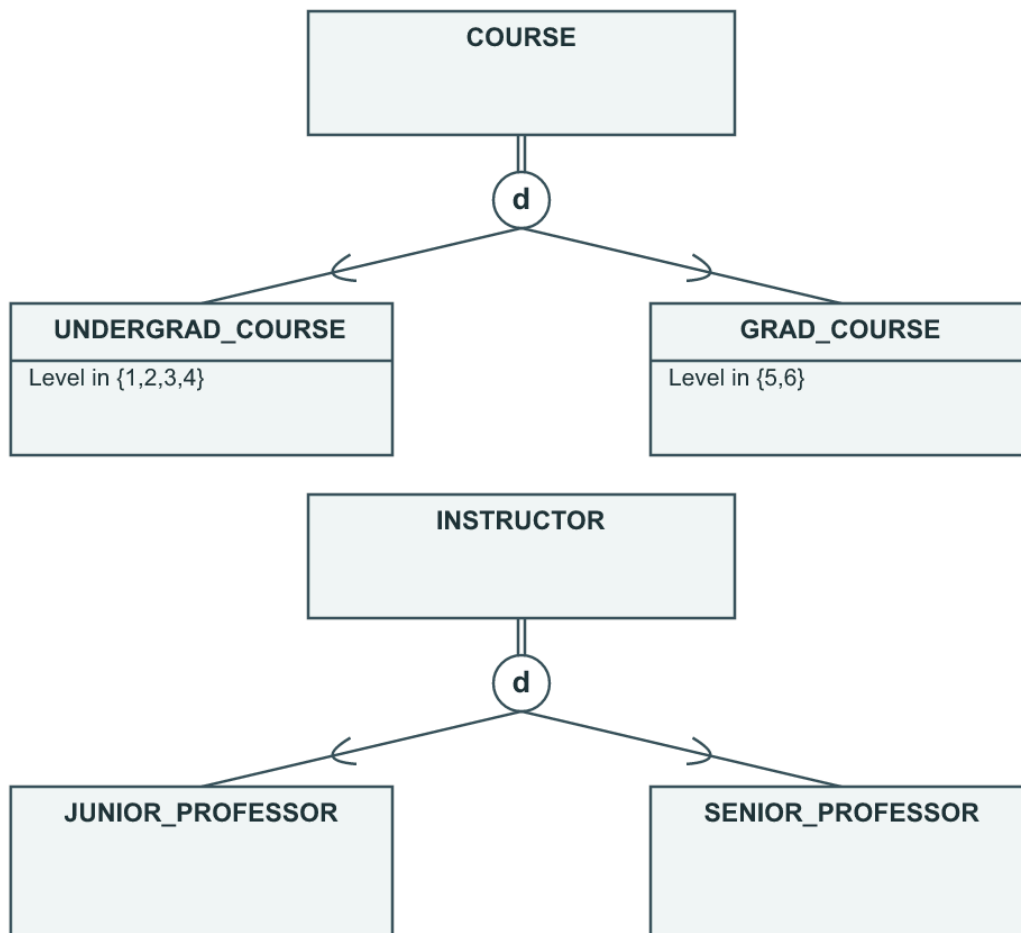
4.33. Undergraduate/Graduate Courses and Instructor Roles

Source: photographed laboratory exercise 4.33, pp.175-176; Chapter 3, Figure 3.20, p.124 and photographed exercise 3.16, pp.126-127. Completion: conceptual modification and constraint examples supplied; modeling-application work not completed.

Start with the Chapter 3 UNIVERSITY model, **not Figure 4.9**. Specialize COURSE into UNDERGRAD_COURSE and GRAD_COURSE, and INSTRUCTOR into JUNIOR_PROFESSOR and SENIOR_PROFESSOR. Both are disjoint, total classifications for this exercise. Keep all inherited identifiers and base relationships; the singular labels here denote the question's plural classes.

Preserve course and instructor identity through specialization

EER



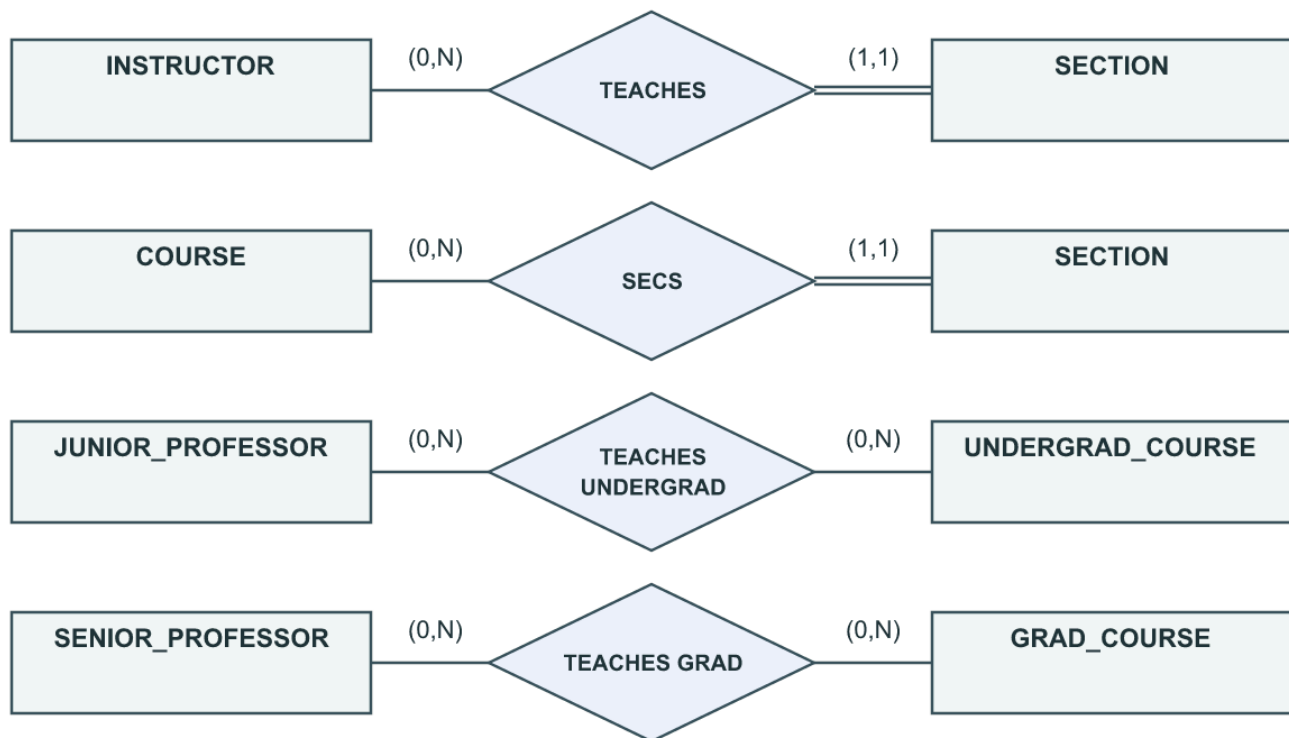
These are the exercise's classifications, not an institution-wide appointment policy.

Class	Inherited key	Example local attributes
UNDERGRAD_COURSE	CCode; CoName is also an independent key in Figure 3.20	GeneralEducationFlag
GRAD_COURSE	CCode; CoName also unique	ResearchComponentDescription
JUNIOR_PROFESSOR	Id	AppointmentStartDate
SENIOR_PROFESSOR	Id	SeniorAppointmentDate

The new attributes are illustrative, not actual institutional promotion rules. COURSE retains Level, Credits and CDesc. INSTRUCTOR retains IName, IOffice, IPhone and Rank. This example defines UNDERGRAD_COURSE by Level in {1,2,3,4} and GRAD_COURSE by Level in {5,6}, matching the base model's level domain. Junior/senior status is explicitly assigned; do not infer it from age.

Section assignments determine the derived course links

ER min-max participation



Last two links are derived from TEACHES + SECS. The cross-relationship level rule is mandatory.

Retain INSTRUCTOR-TEACHES-SECTION and SECTION-SECS-COURSE. Each SECTION has exactly one instructor and one course. Require, for every teaching assignment, either a junior professor and an undergraduate course, or a senior professor and a graduate course. This is a cross-relationship constraint. Do not replace the section assignment by only a course-level link: one course may have several sections taught by different instructors.

To make the requested role-specific links explicit, add **derived** JUNIOR_PROFESSOR-TEACHES_UNDERGRAD-UNDERGRAD_COURSE and SENIOR_PROFESSOR-TEACHES_GRAD-GRAD_COURSE, both (0,N) at each end. A pair exists exactly when at least one SECTION of that course is taught by that instructor. They summarize assignments rather than independently storing contradictory teaching facts. A tool that cannot mark derived relationships must record this derivation as a constraint.

Base type retained	Keys and attributes
COLLEGE	CName key; COffice, CPhone
DEPT	DCode and DName independent keys; DOffice, DPhone
STUDENT	Sid key; SName(FName,MName,LName), Addr, Phone, Major, DOB
SECTION	SecId globally unique; SecNo, Sem, Year, CRoom(Bldg,RoomNo), DaysTime

Retain ADMINS: COLLEGE (0,N), DEPT (1,1); DEAN: COLLEGE (1,1), INSTRUCTOR (0,1); CHAIR: DEPT (1,1), INSTRUCTOR (0,1), with CStartDate; EMPLOYEES: DEPT (0,N), INSTRUCTOR (1,1); HAS: DEPT (0,N), STUDENT (0,1) following Figure 3.20; OFFERS: DEPT (0,N), COURSE (1,1); SECS: COURSE (0,N), SECTION (1,1); TEACHES: INSTRUCTOR (0,N), SECTION (1,1); TAKES: STUDENT (0,N), SECTION (5,N), with Grade. The Chapter 3 prose's required primary department is a known alternative to the figure's HAS minimum zero.

Assignment	Outcome
Junior J1 teaches section S1 of course U1, Level=2	Valid; derived TEACHES_UNDERGRAD(J1,U1) exists.
Senior R1 teaches section S2 of course G1, Level=5	Valid; derived TEACHES_GRAD(R1,G1) exists.
Junior J1 teaches section S3 of G1	Invalid under the exercise's teaching restriction.
A second teacher is added to S1	Invalid even if both are junior.

Keep photographed 3.16's one current Grade per student-section, no simultaneous sections for one student in the same term/time, and one instructor per section. Keep the base's unique course/term/SecNo and no room or instructor time conflicts under the declared time-slot assumptions. Course specializations do not relax these rules. To finish the software lab, enter the two specializations and derived links, record the cross-relationship restriction, and test all four assignments above in the actual modeling tool.

Scope and Remaining Work

Answers 4.1-4.15 cover the review questions, 4.16-4.27 the photographed written exercises, and 4.28-4.33 the conceptual portions of the laboratory exercises. The six labs still require construction and inspection in a modeling application. Open design answers depend on their stated assumptions. No publisher answer manual, chess engine, deployed database or native modeling project is supplied.